

[跳至主要內容](#)

C 程式設計 | CH8 · CH22 · CH24

15 個完整案例 · 16 支獨立程式 · 154 組 C 測試情境

來源：C_Ch8_22_24_M365_Rebuilt.pptx · 138 頁。列印版保留題目、核心解法、完整程式與測試摘要；互動圖解及完整實測文字請在 HTML 查閱。

GCC 與 Clang 各 154/154 通過；NDEBUG 10/10。JavaScript 互動不是 C 執行器。

[跳至主要內容](#)

01 | 反向輸出 10 筆量測值

整合版 10–13 頁；App.12–13

完整題目

依序讀入 10 筆整數量測值，保留原本儲存順序，再由最後一筆往前輸出。不得把題目改成排序。

輸入：標準輸入：恰好 10 個整數，以空白或換行分隔。

輸出：由第 10 筆到第 1 筆輸出；數字間一個空白，最後換行。

限制：每筆介於 -10000 ~ 10000；缺項、非數字或超出範圍即失敗。入門數值文字約定見導讀頁。

範例輸入

```
34 82 49 102 7 94 23 11 50 31
```

範例結果

```
31 50 11 23 94 7 102 49 82 34
```

核心解法

1. 先以正向迴圈填入 $a[0] \sim a[9]$ 。
2. 第二個迴圈從 $i=10$ 遞減，讀取 $a[i-1]$ 。
3. 只改變輸出順序，不交換陣列元素。

易錯提醒：錯誤的起點 $a[10]$ 已越界；最後有效索引是 9。此題不是就地反轉。

HTML 教學補充界線：題目要求恰好 10 筆；這份入門程式只讀取前 10 筆，不會額外檢查尾端是否還有第 11 筆。

互動保留此行為，不把它改稱為嚴格整行解析。

01_reverse.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>
#define N 10

int main(void) {
    int a[N];
    for (int i = 0; i < N; ++i) {
        if (scanf("%d", &a[i]) != 1 ||
            a[i] < -10000 || a[i] > 10000) {
            fputs("Invalid integer input\n", stderr);
            return EXIT_FAILURE;
        }
    }
    for (int i = N; i > 0; --i)
        printf("%d", a[i - 1], i == 1 ? '\n' : ' ');
    跳至主要內容
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 01_reverse.c -o 01_reverse
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
01-01 題目資料	0	PASS	PASS
01-02 含負數與零	0	PASS	PASS
01-03 上下界	0	PASS	PASS
01-04 相同值	0	PASS	PASS
01-05 缺少8筆	1	PASS	PASS
01-06 非數字	1	PASS	PASS
01-07 超出範圍	1	PASS	PASS
01-08 空輸入	1	PASS	PASS

課堂提問

為何輸出迴圈讀 `a[i-1]`，而不是 `a[i]`？

`i` 由元素數量 10 開始；先減 1 才對應最後有效索引 9。 `i>0` 也避免 `unsigned` 倒數下溢。

[跳至主要內容](#)

02 | 用布林陣列偵測重複數字

整合版 14–18 頁；A pp.23–25

完整題目

讀入一個非負整數，從個位數往高位數檢查，找出第一個「再次出現」的數字；完全沒有重複則明確回報。

輸入：標準輸入：一個十進位整數 n 。

輸出：輸出 Repeated digit: d ；若無重複，輸出 No repeated digit。

限制： $0 \leq n \leq 2147483647$ 。 $n=0$ 視為一個數字 0；不保留前導零，負數與錯誤輸入拒絕。

範例輸入

28212

範例結果

Repeated digit: 2

核心解法

1. seen[10] 初值全 false；索引就是數字 0~9。
2. digit=n%10 取得個位數，先檢查 seen[digit]。
3. 未出現則設 true、n/=10；do...while 讓 0 也被處理一次。

易錯提醒：不能先把 seen[digit] 設為 true 再判斷，否則每個數字都會被誤判為重複。

HTML 教學補充界線：互動陣列中的 T/F 是布林狀態的顯示方式，不是對 C bool 物件位元組表示的宣稱。

02_repeated_digits.c · 完整程式

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    bool seen[10] = {false};
    long n;
    int repeated = -1;
    if (scanf("%ld", &n) != 1 ||
        n < 0 || n > 2147483647L) {
        fputs("Expected 0..2147483647\n", stderr);
        return EXIT_FAILURE;
    }
    do {
        int digit = (int)(n % 10);
        if (seen[digit]) {
            repeated = digit;
            break;
        }
        seen[digit] = true;
        n /= 10;
    } while (n > 0);
    if (repeated != -1)
        printf("Repeated digit: %d\n", repeated);
    else
        printf("No repeated digit\n");
    return 0;
}
```

[跳至主要內容](#)

```
        seen[digit] = true;
        n /= 10;
    } while (n > 0);
    if (repeated >= 0)
        printf("Repeated digit: %d\n", repeated);
    else
        puts("No repeated digit");
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 02_repeated_digits.c -o 02_repeated_digits
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
02-01 題目資料	0	PASS	PASS
02-02 無重複	0	PASS	PASS
02-03 單一零	0	PASS	PASS
02-04 零重複	0	PASS	PASS
02-05 前導零不保留	0	PASS	PASS
02-06 最大允許值	0	PASS	PASS
02-07 負數	1	PASS	PASS
02-08 非數字	1	PASS	PASS
02-09 無輸入	1	PASS	PASS

課堂提問

為何 n=0 不使用 while(n>0) 就結束？

題目把 0 視為具有一位數的有效輸入；do...while 至少執行一次，與題意一致。

[跳至主要內容](#)

03 | 班級成績統計與分布

整合版 19–23 頁；A pp.8,26–30 | 新增

完整題目

讀入全班成績，計算總分、平均、最高分、最低分、及格人數，並統計五個成績區間。

輸入：先輸入人數 n ，再輸入 n 個整數成績。

輸出：輸出 count、sum、min、max、average（小數 2 位）、pass 與五個 bins。

限制： $1 \leq n \leq 100$ ； $0 \leq \text{成績} \leq 100$ ；及格為 ≥ 60 。分組：0–59、60–69、70–79、80–89、90–100。

範例輸入

```
5
59 60 75 90 100
```

範例結果

```
count=5 sum=384 min=59 max=100
average=76.80 pass=4
bins=1 1 1 0 2
```

核心解法

1. 先驗證人數與每筆成績，再開始統計。
2. 以第一筆初始化 min/max；用同一趟掃描累加。
3. 100 分併入最後一組；平均前轉為 double。

易錯提醒：sum/n 若先做整數除法再指定給 double，小數已經遺失；需用 (double)sum/n。

HTML 教學補充界線：互動直條圖與逐步累加是本次 HTML 的教學呈現；五個區間及計算規則沿用投影片與完整 C 程式。

03_score_stats.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>
#define CAP 100

int main(void) {
    int score[CAP], bins[5] = {0};
    int n, sum = 0, pass = 0;
    if (scanf("%d", &n) != 1 || n < 1 || n > CAP) {
        fputs("Expected count 1..100\n", stderr);
        return EXIT_FAILURE;
    }

    for (i = 0; i < n; ++i) {
        if (scanf("%d", &score[i]) != 1 ||
            score[i] < 0 || score[i] > 100) {
            fputs("Invalid score\n", stderr);
            return EXIT_FAILURE;
        }
        sum += score[i];
        if (score[i] >= 60) pass++;
        bins[score[i] / 10]++;
    }

    double average = (double)sum / n;

    printf("count=%d sum=%d min=%d max=%d\n", count, sum, min, max);
    printf("average=%.2f pass=%d\n", average, pass);
    printf("bins=%d %d %d %d %d\n", bins[0], bins[1], bins[2], bins[3], bins[4]);
}
```

[跳至主要內容](#)

```
        return EXIT_FAILURE;
    }
}
int low = score[0], high = score[0];
for (int i = 0; i < n; ++i) {
    int s = score[i];
    sum += s;
    if (s < low) low = s;
    if (s > high) high = s;
    if (s >= 60) ++pass;
    int b = s < 60 ? 0 : (s >= 90 ? 4 : s / 10 - 5);
    ++bins[b];
}
printf("count=%d sum=%d min=%d max=%d\n",
       n, sum, low, high);
printf("average=%.2f pass=%d\n", (double)sum / n, pass);
printf("bins=%d %d %d %d %d\n",
       bins[0], bins[1], bins[2], bins[3], bins[4]);
return EXIT_SUCCESS;
}
```

gcc -std=c17 -Wall -Wextra -Wpedantic 03_score_stats.c -o 03_score_stats

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
03-01 題目資料	0	PASS	PASS
03-02 單筆100	0	PASS	PASS
03-03 全部分界	0	PASS	PASS
03-04 容量上限100筆	0	PASS	PASS
03-05 全不及格	0	PASS	PASS
03-06 0人	1	PASS	PASS
03-07 超過容量	1	PASS	PASS
03-08 分數101	1	PASS	PASS
03-09 非數字分數	1	PASS	PASS
03-10 缺一筆	1	PASS	PASS

課堂提問

為何 bins[score/10] 不能直接用在長度 10 的陣列？

100/10=10，會存取第 11 格而越界。本題採明確的五區間映射，把 90~100 合併。

[跳至主要內容](#)

04 | 有先後順序的最大增幅

整合版 24–29 頁；A pp.59–60 | 新增線性解法

完整題目

量測陣列中的位置代表時間。找出 $\max(a[j] - a[i])$ ，且必須滿足 $i < j$ ；比較雙迴圈與單次掃描兩種解法。

輸入：先輸入 n ，再輸入 n 個整數量測值。

輸出：同一行輸出 brute 與 linear 的結果，兩者必須相同。

限制： $2 \leq n \leq 100$ ；數值介於 $-10000 \sim 10000$ 。必須選兩個不同時刻；全程下降時答案可能為負。

範例輸入

```
5
10 15 90 200 110
```

範例結果

```
brute=190 linear=190
```

核心解法

1. 暴力法枚舉所有 $i < j$ ，時間 $O(n^2)$ 。
2. 線性法在處理 j 前，保存 $a[0..j-1]$ 的最小值。
3. 先用當前值計算增幅，再更新最小值；時間 $O(n)$ 、額外空間 $O(1)$ 。

易錯提醒：不加順序限制的 max-min 是另一道題。best=0 會把全下降資料錯答成 0。

HTML 教學補充界線：互動主要逐步呈現線性解法，最後另以暴力枚舉核對；完整 C 原始碼仍保留兩個函式。

04_ordered_difference.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>
#define CAP 100

static long long brute(const int a[], int n) {
    long long best = (long long)a[1] - a[0];
    for (int i = 0; i < n - 1; ++i)
        for (int j = i + 1; j < n; ++j) {
            long long d = (long long)a[j] - a[i];
            if (d > best) best = d;
        }
    return best;
}

static long long linear(const int a[], int n) {
    long long best = a[1] - a[0];
    long long min = a[0];
    for (int j = 1; j < n; ++j) {
        if (a[j] < min) min = a[j];
        else if (a[j] - min > best) best = a[j] - min;
    }
    return best;
}
```

[跳至主要內容](#)

```
        long long d = (long long)a[j] - min_before;
        if (d > best) best = d;
        if (a[j] < min_before) min_before = a[j];
    }
    return best;
}

int main(void) {
    int a[CAP], n;
    if (scanf("%d", &n) != 1 || n < 2 || n > CAP) {
        fputs("Expected count 2..100\n", stderr);
        return EXIT_FAILURE;
    }
    for (int i = 0; i < n; ++i) {
        if (scanf("%d", &a[i]) != 1 ||
            a[i] < -10000 || a[i] > 10000) {
            fputs("Invalid sample\n", stderr);
            return EXIT_FAILURE;
        }
    }
    printf("brute=%lld linear=%lld\n", brute(a, n), linear(a, n));
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 04_ordered_difference.c -o 04_ordered_difference
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
04-01 題目資料	0	PASS	PASS
04-02 全程下降	0	PASS	PASS
04-03 最低值在最高值之後	0	PASS	PASS
04-04 兩個時刻	0	PASS	PASS
04-05 全部相同	0	PASS	PASS
04-06 交錯正負	0	PASS	PASS
04-07 人數不足	1	PASS	PASS
04-08 容量超界	1	PASS	PASS
04-09 值超界	1	PASS	PASS
04-10 缺值	1	PASS	PASS

課堂提問

為何要先算差值，再更新 min_before ？

計算時 min before 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

[跳至主要內容](#)

05 | 三台設備 × 四時段產量表

整合版 32–36 頁；A pp.36–46 | 新增

完整題目

用二維陣列保存三台設備在四個時段的產量，計算每台設備總產量、每時段總產量與全部總量。

輸入：依列輸入 12 個整數：先輸入設備 0 的四時段，再設備 1、設備 2。

輸出：輸出 rows 三筆、cols 四筆，以及 total。

限制：固定 3 列 × 4 欄；每格 0 ~ 10000。列代表設備，欄代表時段；不可交換索引意義。

範例輸入

```
10 20 30 40
5 15 25 35
8 12 16 24
```

範例結果

```
rows=100 80 60
cols=23 47 71 99
total=240
```

核心解法

1. 以 `qty[r][c]` 同時表達設備與時段。
2. 外層走設備 `r`，內層走時段 `c`。
3. 每讀取一格，加入 `rows[r]`、`cols[c]` 與 `total`。

易錯提醒：`qty[r,c]` 使用的是逗號運算子，不是二維索引；正確寫法必須是 `qty[r][c]`。

HTML 教學補充界線：投影片解法摘要寫「每讀取一格，加入總量」；完整 C 程式實際分成「先讀完」與「再統計」兩組迴圈。本 HTML 保留摘要原文，逐步圖解則依完整程式的實際順序。

05_matrix_report.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>
#define ROWS 3
#define COLS 4

int main(void) {
    int qty[ROWS][COLS];
    long rows[ROWS] = {0}, cols[COLS] = {0}, total = 0;
    for (int r = 0; r < ROWS; ++r)
        for (int c = 0; c < COLS; ++c) {
            printf("%d", &qty[r][c]) != 1 ||
            跳至主要內容 [r][c] < 0 || qty[r][c] > 10000) {
                puts("Invalid quantity\n", stderr);
                return EXIT_FAILURE;
            }
            rows[r] += qty[r][c];
            cols[c] += qty[r][c];
            total += qty[r][c];
        }
    for (int r = 0; r < ROWS; ++r)
        printf("rows=%ld ", rows[r]);
    for (int c = 0; c < COLS; ++c)
        printf("cols=%ld ", cols[c]);
    printf("total=%ld\n", total);
    return 0;
}
```

```
        }
    }
    for (int r = 0; r < ROWS; ++r)
        for (int c = 0; c < COLS; ++c) {
            rows[r] += qty[r][c];
            cols[c] += qty[r][c];
            total += qty[r][c];
        }
    printf("rows=%ld %ld %ld\n", rows[0], rows[1], rows[2]);
    printf("cols=%ld %ld %ld %ld\n",
           cols[0], cols[1], cols[2], cols[3]);
    printf("total=%ld\n", total);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 05_matrix_report.c -o 05_matrix_report
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
05-01 題目資料	0	PASS	PASS
05-02 全零	0	PASS	PASS
05-03 每格10000	0	PASS	PASS
05-04 row-major順序	0	PASS	PASS
05-05 負產量	1	PASS	PASS
05-06 超範圍	1	PASS	PASS
05-07 缺11筆	1	PASS	PASS
05-08 非數字	1	PASS	PASS

課堂提問

為何 rows 和 cols 的陣列長度不同？

rows 為每台設備保存一個結果，共 3 筆；cols 為每個時段保存一個結果，共 4 筆。

[跳至主要內容](#)

06 | 五種利率的逐年複利表

整合版 37–41 頁；App.31–35

完整題目

以本金 100 作為數值運算練習，比較起始利率及其後四個整數百分比利率，列出逐年複利後的金額。

輸入：標準輸入：起始年利率 `rate`（整數百分比）與年數 `years`。

輸出：輸出 5 個利率欄，每年一行，金額顯示小數 2 位。

限制： $0 \leq \text{rate} \leq 20$ ； $1 \leq \text{years} \leq 30$ 。每年複利一次；本金固定 100。不涉及費用、稅務或投資建議。

範例輸入

6 5

範例結果

第 5 年：
133.82 140.26 146.93 153.86 161.05

核心解法

1. `value[5]` 分別保存五種情境的「當年餘額」。
2. 外層迴圈推進年份；內層更新每個利率。
3. `value[i] *= 1.0 + (rate+i)/100.0`；只保存最近一年的狀態。

易錯提醒：除以 100 會做整數除法；此題必須除以 100.0。顯示四捨五入不代表中間值已被截斷。

HTML 教學補充界線：這是陣列與巢狀迴圈的數值題，不是金融試算建議。互動以 JavaScript 浮點推演；下方 C 實測保留真正 `printf` 的字距及小數格式。

06_interest_table.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>
#define RATES 5

int main(void) {
    int rate, years;
    double value[RATES];
    if (scanf("%d%d", &rate, &years) != 2 ||
        rate < 0 || rate > 20 || years < 1 || years > 30) {
        fputs("Expected rate 0..20 and years 1..30\n", stderr);
        return EXIT_FAILURE;
    }
    printf("Year");
    for (i = 0; i < RATES; ++i) {
        printf(" %10.2f", 100.0);
        if (i % 4 == 3)
            printf("\n");
        else
            printf(" ");
        value[i] = 100.0;
        for (year = 1; year <= years; ++year) {
            value[i] *= 1.0 + (rate + i) / 100.0;
        }
        printf(" %10.2f", value[i]);
        if (i % 4 == 3)
            printf("\n");
        else
            printf(" ");
    }
    printf("\n");
}
```

```
    putchar('\n');
    for (int year = 1; year <= years; ++year) {
        printf("%4d", year);
        for (int i = 0; i < RATES; ++i) {
            value[i] *= 1.0 + (rate + i) / 100.0;
            printf(" %8.2f", value[i]);
        }
        putchar('\n');
    }
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 06_interest_table.c -o 06_interest_table
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
06-01 題目利率	0	PASS	PASS
06-02 零起始利率	0	PASS	PASS
06-03 上限情境	0	PASS	PASS
06-04 負利率	1	PASS	PASS
06-05 超過利率	1	PASS	PASS
06-06 零年	1	PASS	PASS
06-07 超過年數	1	PASS	PASS
06-08 缺少年數	1	PASS	PASS

課堂提問

為何不需要保存 years×5 的整張表？

下一年只依賴上一年的餘額，已經輸出的歷史資料不再參與計算，因此五個狀態值就足夠。

[跳至主要內容](#)

07 | 不重複發牌與狀態表

整合版 42–46 頁；A pp.49–55

完整題目

從 4 種花色、13 種點數的牌組發出 n 張不重複的牌，用二維布林表記錄已發出的牌。

輸入：標準輸入：張數 n 與 unsigned 整數亂數種子 `seed`。

輸出：輸出 n 個二元素牌碼，例如 `7c`；牌碼間以空白分隔。 $n=0$ 只輸出換行。

限制： $0 \leq n \leq 52$ 。點數 `23456789tjqka`；花色 `cdhs`。同一實作 / 種子可重現；跨編譯器不保證相同牌序。

範例輸入

5 1

範例結果

5 張合法、互不重複的牌
(精確序列依 C 函式庫而異)

核心解法

- 1. `rand()%4` 與 `rand()%13` 取得花色、點數。
- 2. `in_hand[suit][rank]` 已為 `true` 就重新抽。
- 3. 新牌才標記、輸出並增加 `dealt`；查表轉成牌碼。

易錯提醒：`rand()%n` 可能有模數偏差；這是陣列與狀態管理練習，不是公平抽獎或密碼用途的亂數設計。

HTML 教學補充界線：HTML 使用 `xorshift32` 產生可觀察的教學牌序，不模仿 C 函式庫 `rand`。C 測試驗證張數、牌碼合法與不重複，不固定跨平台的精確牌序。

07_deal_cards.c · 完整程式

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#define SUITS 4
#define RANKS 13

int main(void) {
    const char ranks[] = "23456789tjqka";
    const char suits[] = "cdhs";
    bool in_hand[SUITS][RANKS] = {{false}};
    int n;
    unsigned seed;
    if (scanf("%d%u", &n, &seed) != 2 || n < 0 || n > 52) {
        printf("Invalid count 0..52 and unsigned seed\n", stderr);
        return EXIT_FAILURE;
    }
    srand(seed);
```

```
int dealt = 0;
while (dealt < n) {
    int suit = rand() % SUITS;
    int rank = rand() % RANKS;
    if (in_hand[suit][rank]) continue;
    in_hand[suit][rank] = true;
    printf("%c%c%s", ranks[rank], suits[suit],
           dealt + 1 == n ? "" : " ");
    ++dealt;
}
putchar('\n');
return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 07_deal_cards.c -o 07_deal_cards
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
07-01 零張	0	PASS	PASS
07-02 題目五張種子1	0	PASS	PASS
07-03 五張種子42	0	PASS	PASS
07-04 全副52張	0	PASS	PASS
07-05 單張	0	PASS	PASS
07-06 另一種子	0	PASS	PASS
07-07 超過52張	1	PASS	PASS
07-08 負張數	1	PASS	PASS
07-09 缺少種子	1	PASS	PASS
07-10 非數字	1	PASS	PASS

課堂提問

發到重複牌時，能否把 dealt 加一？

不可以。dealt 代表成功輸出的不同牌張數，而不是抽樣嘗試次數。只有確認新牌後才增加。

[跳至主要內容](#)

08 | 命令列檔案開啟檢查器

整合版 55–59 頁；B pp.38–42 | C pp.6–9

完整題目

寫一個命令列工具，接收一個檔名，檢查本次是否能以讀取模式成功開啟並關閉。

輸入：命令列：canopen filename；不從鍵盤另外讀取檔名。

輸出：成功：stdout 顯示檔案可開啟。失敗：stderr 顯示原因，回傳 EXIT_FAILURE。

限制：argc 必須為 2；以本次 fopen 的結果為準。可開啟不代表內容正確，也不保證未來仍可開啟。

範例輸入

```
canopen sample.txt
```

範例結果

```
sample.txt can be opened
```

核心解法

1. 先驗證 argc，再使用 argv[1]。
2. fopen 回傳 NULL 就停止；保存 errno 後才輸出訊息。
3. 成功開啟也要檢查 fclose；不要把錯誤訊息寫入 stdout。

易錯提醒：先用「檔案是否存在」判斷再 fopen 仍可能失敗；權限與狀態可能改變，應檢查真正要做的操作。

HTML 教學補充界線：互動的「檔案存在 / 缺少 / 關閉失敗」是虛擬情境，不會存取電腦路徑。作業系統的 perror 字串不被當作固定答案。

08_canopen.c · 完整程式

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fputs("usage: canopen filename\n", stderr);
        return EXIT_FAILURE;
    }
    errno = 0;
    FILE *fp = fopen(argv[1], "r");
    if (fp == NULL) {
        int saved_errno = errno;
        fprintf(stderr, "Cannot open: %s\n", argv[1]);
        if (errno != 0) {
跳至主要內容 : saved_errno;
        }
        return EXIT_FAILURE;
    }
    fclose(fp);
    return EXIT_SUCCESS;
}
```

```
        return EXIT_FAILURE;
    }
    if (fclose(fp) != 0) {
        fputs("Close failed\n", stderr);
        return EXIT_FAILURE;
    }
    printf("%s can be opened\n", argv[1]);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 08_canopen.c -o 08_canopen
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
08-01 文字檔可開啟	0	PASS	PASS
08-02 空檔也可開啟	0	PASS	PASS
08-03 檔案不存在	1	PASS	PASS
08-04 缺少檔名	1	PASS	PASS
08-05 多一個參數	1	PASS	PASS

課堂提問

argc 為 2 時，argv[0] 與 argv[1] 分別是什麼？

argv[0] 是程式名稱字串，argv[1] 是使用者傳入的檔名；argv[argc] 是空指標。

[跳至主要內容](#)

09 | 日期字串：格式合法不等於日期存在

整合版 68–73 頁；B pp.132–135,161–171 | 新增

完整題目

讀入一行日期，接受 YYYY-MM-DD 或 YYYY/MM/DD，驗證實際日曆日期後，統一輸出 YYYY-MM-DD。

輸入：一行 10 個字元的日期；允許行尾 LF、CRLF 或最後一行無換行。

輸出：有效日期：輸出正規化結果；無效：stderr 訊息並失敗。

限制：1900~2099 年；年月日必須分別為 4/2/2 位數；兩個分隔符須相同。不能有多餘字元或空白。輸入為無 NUL 的文字。

範例輸入

```
2024/02/29
```

範例結果

```
2024-02-29
```

核心解法

1. fgets 取得一整行，只移除行尾 CR/LF。
2. 先驗證長度與數字位置，再以 sscanf 拆欄位。
3. 檢查月份、閏年與當月天數，最後 snprintf 並檢查截斷。

易錯提醒：只檢查 sscanf 回傳 3 或 5，只能證明欄位解析成功，不能證明日期在日曆中存在。

HTML 教學補充界線：日期圖解沿用 1900~2099 年及固定 4/2/2 位格式，不擴充成任意地區的日期解析器。

09_date_parser.c · 完整程式

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char line[64], sep1, sep2, extra;
    int y, m, d;
    if (fgets(line, sizeof line, stdin) == NULL) {
        fputs("Missing date\n", stderr);
        return EXIT_FAILURE;
    }
    size_t len = strlen(line);
    if (len > 0 && line[len - 1] == '\n') --len;
    if (len > 0 && line[len - 1] == '\r') --len;
    if (!is_valid_date(line, &y, &m, &d)) {
        fputs("Invalid date: expected YYYY-MM-DD or YYYY/MM/DD\n", stderr);
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

```
    }
    line[len] = '\0';
    for (size_t i = 0; i < len; ++i) {
        if (i != 4 && i != 7 &&
            !isdigit((unsigned char)line[i])) {
            fputs("Invalid date digits\n", stderr);
            return EXIT_FAILURE;
        }
    }
    if (sscanf(line, "%4d%c%2d%c%2d%c",
               &y, &sep1, &m, &sep2, &d, &extra) != 5 ||
        sep1 != sep2 || (sep1 != '-' && sep1 != '/') ||
        y < 1900 || y > 2099 || m < 1 || m > 12) {
        fputs("Invalid date format/range\n", stderr);
        return EXIT_FAILURE;
    }
    const int days[] = {31,28,31,30,31,30,31,31,30,31,30,31};
    int leap = y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
    int limit = days[m - 1] + (m == 2 && leap);
    if (d < 1 || d > limit) {
        fputs("Invalid day\n", stderr);
        return EXIT_FAILURE;
    }
    char out[11];
    int written = snprintf(out, sizeof out, "%04d-%02d-%02d", y, m, d);
    if (written < 0 || (size_t)written >= sizeof out) {
        fputs("Formatting failed\n", stderr);
        return EXIT_FAILURE;
    }
    puts(out);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 09_date_parser.c -o 09_date_parser
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
09-01 閏年斜線	0	PASS	PASS
09-02 400年規則	0	PASS	PASS
09-03 年份上界	0	PASS	PASS
09-04 沒有末尾換行	0	PASS	PASS
09-05 CRLF結尾	0	PASS	PASS
09-06 平年2月29	1	PASS	PASS
09-07 1900不是閏年	1	PASS	PASS
09-08 4月31日	1	PASS	PASS
09 00 公區符運用	1	PASS	PASS
09 跳至主要內容	1	PASS	PASS

情境	預期返回碼	GCC	Clang
09-11 月份13	1	PASS	PASS
09-12 尾端垃圾	1	PASS	PASS
09-13 非數字日期	1	PASS	PASS
09-14 空輸入	1	PASS	PASS

課堂提問

為何本題先檢查數字位置，再呼叫 sscanf ？

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

[跳至主要內容](#)

10 | 安全複製：逐字元版與區塊版

整合版 76–84 頁；B pp.117–128,138–146 | 新增防護

完整題目

複製任意文字或二進位檔案，保留所有位元組。實作逐字元與區塊兩版，且不得覆寫任何既有目的檔。

輸入：命令列：copy_character source new_destination，或 copy_block source new_destination。

輸出：成功：目的檔每個位元組與來源一致，stdout 顯示 Copy complete。

限制：目的檔必須不存在；使用 C17 的 wbx 建立。檢查讀、寫、關閉錯誤；失敗時嘗試刪除本次建立的不完整檔。

範例輸入

```
copy_block input.bin copy.bin
```

範例結果

```
Copy complete
copy.bin 與 input.bin 位元組完全一致
```

核心解法

1. 來源用 rb；目的用 wbx，避免既有檔案被截斷。
2. 逐字元版：int 接 fgetc；區塊版：以 n 傳遞有效長度。
3. 結束後查 ferror 與 fclose；任何失敗都不能宣稱複製完成。

易錯提醒：若使用 wb，開檔當下就可能毀掉原內容。fwrite 必須寫本次 fread 回傳的 n，而不是固定 4096。

HTML 教學補充界線：本區行號解說主要對應 10b 區塊版；兩份完整程式都可切換查看。互動故障注入為教學模型，與測試紀錄中的實際檔案操作分開列示。

10a_copy_character.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fputs("usage: copy_character source new_destination\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *in = fopen(argv[1], "rb");
    if (in == NULL) {
        fputs("Cannot open source\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *out = fopen(argv[2], "wbx");
    if (out == NULL) {
```

```

        fputs("Destination exists or cannot be created\n", stderr);
        fclose(in);
        return EXIT_FAILURE;
    }
    int ch, failed = 0;
    while ((ch = fgetc(in)) != EOF) {
        if (fputc(ch, out) == EOF) {
            failed = 1;
            break;
        }
    }
    if (ferror(in)) failed = 1;
    if (fclose(in) != 0) failed = 1;
    if (fclose(out) != 0) failed = 1;
    if (failed) {
        fputs("Copy failed; removing partial output\n", stderr);
        if (remove(argv[2]) != 0)
            fputs("Warning: partial output remains\n", stderr);
        return EXIT_FAILURE;
    }
    puts("Copy complete");
    return EXIT_SUCCESS;
}

```

gcc -std=c17 -Wall -Wextra -Wpedantic 10a_copy_character.c -o 10a_copy_character

10b_copy_block.c · 完整程式

```

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fputs("usage: copy_block source new_destination\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *in = fopen(argv[1], "rb");
    if (in == NULL) {
        fputs("Cannot open source\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *out = fopen(argv[2], "wbx");
    if (out == NULL) {
        fputs("Destination exists or cannot be created\n", stderr);
        fclose(in);
        return EXIT_FAILURE;
    }
    unsigned char buffer[4096];
    size_t n;
    int failed = 0;
    while ((n = fread(buffer, 1, sizeof buffer, in)) > 0) {
        if (fwrite(buffer, 1, n, out) != n) {
            failed = 1;
            break;
        }
    }

    跳至主要內容
    failed = 1;
    != 0) failed = 1;
    if (fclose(out) != 0) failed = 1;

```

```
    if (failed) {
        fputs("Copy failed; removing partial output\n", stderr);
        if (remove(argv[2]) != 0)
            fputs("Warning: partial output remains\n", stderr);
        return EXIT_FAILURE;
    }
    puts("Copy complete");
    return EXIT_SUCCESS;
}
```

gcc -std=c17 -Wall -Wextra -Wpedantic 10b_copy_block.c -o 10b_copy_block

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
10-01 a版：含00/1A/FF	0	PASS	PASS
10-02 a版：空檔	0	PASS	PASS
10-03 a版：ASCII文字	0	PASS	PASS
10-04 a版：跨越4096邊界	0	PASS	PASS
10-05 a版：目的已存在	1	PASS	PASS
10-06 a版：來源不存在	1	PASS	PASS
10-07 a版：來源目的相同	1	PASS	PASS
10-08 a版：缺少參數	1	PASS	PASS
10-09 b版：含00/1A/FF	0	PASS	PASS
10-10 b版：空檔	0	PASS	PASS
10-11 b版：ASCII文字	0	PASS	PASS
10-12 b版：跨越4096邊界	0	PASS	PASS
10-13 b版：目的已存在	1	PASS	PASS
10-14 b版：來源不存在	1	PASS	PASS
10-15 b版：來源目的相同	1	PASS	PASS
10-16 b版：缺少參數	1	PASS	PASS

課堂提問

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

[跳至主要內容](#)

11 | 直接更新庫存檔的第 3 筆

整合版 89–94 頁；B pp.138–160 | 原庫存案例改編

完整題目

建立五筆整數庫存的二進位檔，讀出索引 2 的舊值，再直接改成 99，最後重讀整個檔案確認結果。

輸入：命令列：stock_update new_binary_file。初始資料固定為 10、20、35、40、50。

輸出：顯示 old=35 new=99，及更新後的五筆整數。

限制：目的檔必須不存在；第 3 筆 = 索引 2。原生 int 格式僅限相同實作讀寫；這是定位練習，不提供交易式還原。

範例輸入

```
stock_update stock.bin
```

範例結果

```
old=35 new=99
10 20 99 40 50
```

核心解法

1. 先 wbx 寫入初始陣列，關閉後以 rb+ 重開。
2. 位置 = 2×sizeof(int)；先 fseek 再讀出舊值。
3. 讀→寫前重新定位；寫→讀前再定位到檔首。

易錯提醒：第 k 筆以 0-based 索引 k 計算 offset；更換平台不能假設 int 永遠是 4 bytes，也不能忽略位元組順序。

HTML 教學補充界線：sizeof(int) 的切換只改變圖解的位移假設，不改動固定 C 程式，更不宣稱原生 int 二進位資料可跨平台交換。

11_stock_update.c · 完整程式

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int stock[5] = {10, 20, 35, 40, 50};
    int updated = 99, previous;
    int failed = 0;
    if (argc != 2) {
        fputs("usage: stock_update new_binary_file\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *fp = fopen(argv[1], "wbx");
    if (!fp) {
        fputs("cannot create new stock file\n", stderr);
        return EXIT_FAILURE;
    }
    fwrite(stock, sizeof(int), 5, fp);
    fclose(fp);
    fp = fopen(argv[1], "rb+");
    if (!fp) {
        fputs("cannot open stock file\n", stderr);
        return EXIT_FAILURE;
    }
    previous = 35;
    fseek(fp, 2 * sizeof(int), SEEK_SET);
    previous = fgetc(fp);
    if (previous != 35) {
        fputs("previous value is not 35\n", stderr);
        return EXIT_FAILURE;
    }
    fputc(99, fp);
    fseek(fp, 0, SEEK_SET);
    while (fgetc(fp) != 10)
        continue;
    while (fgetc(fp) != 20)
        continue;
    while (fgetc(fp) != 99)
        continue;
    while (fgetc(fp) != 40)
        continue;
    while (fgetc(fp) != 50)
        continue;
    return EXIT_SUCCESS;
}
```

```
    }
    if (fwrite(stock, sizeof stock[0], 5, fp) != 5) failed = 1;
    if (fclose(fp) != 0) failed = 1;
    if (failed) {
        fputs("Initial write failed\n", stderr);
        return EXIT_FAILURE;
    }
    fp = fopen(argv[1], "rb+");
    if (fp == NULL) {
        fputs("Cannot reopen stock file\n", stderr);
        return EXIT_FAILURE;
    }
    long offset = 2L * (long)sizeof stock[0];
    if (fseek(fp, offset, SEEK_SET) != 0 ||
        fread(&previous, sizeof previous, 1, fp) != 1 ||
        fseek(fp, offset, SEEK_SET) != 0 ||
        fwrite(&updated, sizeof updated, 1, fp) != 1 ||
        fseek(fp, 0L, SEEK_SET) != 0 ||
        fread(stock, sizeof stock[0], 5, fp) != 5) {
        fputs("Record update/read failed\n", stderr);
        failed = 1;
    }
    if (fclose(fp) != 0) failed = 1;
    if (failed) return EXIT_FAILURE;
    printf("old=%d new=%d\n", previous, stock[2]);
    for (int i = 0; i < 5; ++i)
        printf("%d%c", stock[i], i == 4 ? '\n' : ' ');
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 11_stock_update.c -o 11_stock_update
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
11-01 建立與定位更新	0	PASS	PASS
11-02 拒絕既有檔案	1	PASS	PASS
11-03 父目錄不存在	1	PASS	PASS
11-04 缺少檔名	1	PASS	PASS

課堂提問

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

[跳至主要內容](#)

12 | 嚴格讀取一個 0~100 分的整數

整合版 100–105 頁；B pp.103–104,132–135 | C pp.6–9 | 新增

完整題目

讀入一行成績，不只判斷是否以數字開頭，還要拒絕超大整數、尾端垃圾、空白行與超長輸入。

輸入：標準輸入的一行無 NUL 文字；允許數字前後空白及正號。

輸出：有效：score=value。無效：stderr 回報 No number、Trailing text、Out of range 等。

限制：數值 0~100；單行最多 126 bytes（不含 LF）；不接受小數或第二個值。最末行可不含換行。

範例輸入

```
+86
```

範例結果

```
score=86
```

核心解法

1. fgets 限制讀取容量；過長輸入整行拒絕。
2. errno=0 後 strtol，保留 end 指標。
3. 依序檢查沒讀到數字、ERANGE / 範圍、尾端非空白；成功才設定輸出值。

易錯提醒：atoi 無法清楚區分「合法 0」與轉換失敗；只看 errno 也無法偵測 86abc。

HTML 教學補充界線：來源以一行無 NUL 文字為約定，最多 126 bytes 不含 LF。互動使用 BigInt 判斷巨大整數；不把 JavaScript 數值範圍當成 C long 的範圍。

12_strict_score.c · 完整程式

```
#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum Status { OK, NO_NUMBER, TRAILING, OUT_OF_RANGE };

static enum Status parse_score(const char *text, int *out) {
    char *end;
    errno = 0;
    long value = strtol(text, &end, 10);
    if (end == text) return NO_NUMBER;
    if (errno == ERANGE || value < 0 || value > 100)
        return OUT_OF_RANGE;
    if (!isspace(*end)) return TRAILING;
    *out = (int)value;
    return OK;
}
```

[跳至主要內容](#)

```
        return OK;
    }

    int main(void) {
        char line[128];
        int score;
        if (fgets(line, sizeof line, stdin) == NULL) {
            fputs(ferror(stdin) ? "Read error\n" : "No input\n", stderr);
            return EXIT_FAILURE;
        }
        if (strchr(line, '\n') == NULL && !feof(stdin)) {
            fputs("Line too long (max 126 bytes)\n", stderr);
            return EXIT_FAILURE;
        }
        enum Status status = parse_score(line, &score);
        if (status != OK) {
            const char *message[] = {
                "OK", "No number", "Trailing text", "Out of range"
            };
            fprintf(stderr, "%s\n", message[status]);
            return EXIT_FAILURE;
        }
        printf("score=%d\n", score);
        return EXIT_SUCCESS;
    }
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 12_strict_score.c -o 12_strict_score
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
12-01 題目空白正號	0	PASS	PASS
12-02 零分	0	PASS	PASS
12-03 一百分	0	PASS	PASS
12-04 無末尾換行	0	PASS	PASS
12-05 126bytes合法行	0	PASS	PASS
12-06 126bytes無換行	0	PASS	PASS
12-07 尾端文字	1	PASS	PASS
12-08 兩個值	1	PASS	PASS
12-09 小數	1	PASS	PASS
12-10 負數	1	PASS	PASS
12-11 大於100	1	PASS	PASS
12-12 60位大整數	1	PASS	PASS
12 跳至主要內容	1	PASS	PASS
12-14 六分正號	1	PASS	PASS

情境	預期返回碼	GCC	Clang
12-15 127bytes行	1	PASS	PASS
12-16 127bytes無換行	1	PASS	PASS
12-17 空輸入	1	PASS	PASS

課堂提問

為何 `*out=(int)value` 要放在所有檢查之後？

只有值已落在 `0 ~ 100`，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

[跳至主要內容](#)

13 | 外部驗證與 assert 的分工

整合版 107–110 頁；C pp.3–5 | 新增防護

完整題目

讀入數值後計算平方根。外部不合法輸入必須以正常錯誤路徑拒絕；assert 僅記錄驗證後的內部不變量。

輸入：標準輸入：一個可由 double 表示的數值文字。

輸出：有效：輸出 sqrt=結果（小數 6 位）；無效：stderr 訊息並失敗。

限制：只接受有限且非負的值；NaN、正負無窮大與負數拒絕。定義 NDEBUG 後，外部驗證仍須有效。

範例輸入

4

範例結果

sqrt=2.000000

核心解法

1. if 檢查轉換結果、isfinite(x) 與 $x \geq 0$ 。
2. assert($x \geq 0$) 說明程式到這裡應滿足的條件。
3. 用一般版與 -DNDEBUG 版跑同一批測試。

易錯提醒：不可寫 `assert(scanf(...)=1)`；NDEBUG 啟用時，連讀取輸入的副作用都可能消失。

HTML 教學補充界線：NDEBUG 開關是圖解；測試區另附真正以 -DNDEBUG 編譯後的 10 組結果。互動平方根只示範十進位格式與有限顯示範圍，完整 C 程式以來源規格為準。

13_assert_sqrt.c · 完整程式

```
#include <assert.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    double x;
    if (scanf("%lf", &x) != 1 || !isfinite(x) || x < 0.0) {
        fputs("Expected a finite, nonnegative value\n", stderr);
        return EXIT_FAILURE;
    }
    assert(x >= 0.0); /* Internal invariant; no side effects. */
    printf("sqrt=%.6f\n", sqrt(x));
    return EXIT_SUCCESS;
}
```

[跳至主要內容](#)

gcc -std=c17 -Wall -Wextra -Wpedantic 13_assert_sqrt.c -o 13_assert_sqrt -lm

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
13-01 四的平方根	0	PASS	PASS
13-02 零	0	PASS	PASS
13-03 非整數平方根	0	PASS	PASS
13-04 小數	0	PASS	PASS
13-05 負數	1	PASS	PASS
13-06 NaN	1	PASS	PASS
13-07 正無窮	1	PASS	PASS
13-08 負無窮	1	PASS	PASS
13-09 非數字	1	PASS	PASS
13-10 空輸入	1	PASS	PASS

課堂提問

assert 關閉之後，這個程式會接受負數嗎？

不會。外部驗證在 if 中，與 assert 是否啟用無關；assert 只是協助檢查內部邏輯。

[跳至主要內容](#)

14 | 觀察 log 的 errno 診斷

整合版 112–116 頁；C pp.6–9 | 勘誤 / 補充

完整題目

依序計算 $\log(-1)$ 、 $\log(0)$ 、 $\log(1)$ ，觀察數學函式的結果與錯誤分類，不把平台輸出字串當成固定答案。

輸入：無外部輸入；程式內的 input 陣列固定為 -1 、 0 、 1 。

輸出：先印 MATH_ERRNO 是否支援；逐筆印結果與 EDOM、ERANGE、none，或不支援 errno 檢查。

限制：只有 math_errhandling 含 MATH_ERRNO 時，才以 errno 驗證錯誤分類。NaN/inf 文字與 strerror 訊息依實作。

範例輸入

直接執行 math_errno

範例結果

支援 MATH_ERRNO 時：
 $-1 \rightarrow \text{EDOM}$ ； $0 \rightarrow \text{ERANGE}$ ； $1 \rightarrow \text{none}$

核心解法

1. 先確認數學函式的錯誤回報機制。
2. 每次呼叫 log 前把 errno 設為 0。
3. log 後立即保存 errno，再交給 printf / strerror 產生說明。

易錯提醒：原稿的 1.0/0.0 不是會替你設定 errno 的函式呼叫；不能把它當作可攜的 ERANGE 示範。

HTML 教學補充界線：HTML 的 NaN、 $-\text{inf}$ 是語意圖解。真正 C 輸出的大小寫、負號、錯誤碼數值與語系，以各次實測紀錄為準。

14_math_errno.c · 完整程式

```
#include <errno.h>
#include <math.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    const double input[] = {-1.0, 0.0, 1.0};
    int has_errno = (math_errhandling & MATH_ERRNO) != 0;
    printf("MATH_ERRNO=%d\n", has_errno);
    for (size_t i = 0; i < sizeof input / sizeof input[0]; ++i) {
        errno = 0;
        double result = log(input[i]);
        printf("%s: log(%g) = %g, errno = %d\n",
            i < 2 ? "跳至主要內容" : "", input[i], result,
            !has_errno ? 0 : errno);
    }
    return !has_errno ? 0 : 1;
}
```

```
        puts("errno checking not supported");
    else if (saved_errno == 0)
        puts("error=none");
    else {
        const char *name = saved_errno == EDOM ? "EDOM" :
                            saved_errno == ERANGE ? "ERANGE" : "OTHER";
        printf("error=%s (%s)\n", name, strerror(saved_errno));
    }
}
return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 14_math_errno.c -o 14_math_errno -lm
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
14-01 三個log結果與errno分類	0	PASS	PASS

課堂提問

如果不在每次 log 前清除 errno，會發生什麼誤判？

先前失敗留下的非零值可能仍存在，後續成功呼叫就會被誤認為失敗；仍須遵守特定函式的規格。

[跳至主要內容](#)

15 | 整合專題：成績檔驗證與報表

整合版 119~130 頁；A+B+C | 新增整合實作

完整題目

從成績文字檔載入資料，全部驗證通過後才建立統計報表；任一筆不合法就停止，不產生看似成功的部分報表。

輸入：命令列：grade_file input_csv new_report。每行 Sddd,score，沒有標題列。

輸出：報表第一行列出 count、average、min、max、pass；後續保留輸入順序，逐列輸出 ID、成績、PASS/FAIL。

限制：1~100 筆；ID 為 S000~S999 且不可重複；成績 0~100。無 BOM/NUL 的文字檔；禁止空白行；目的報表必須不存在。

範例輸入

```
S001,86
S002,100
S003,59
S004,75
```

範例結果

```
count=4 average=80.00 min=59 max=100 pass=3
S001,86,PASS
S002,100,PASS
S003,59,FAIL
S004,75,PASS
```

核心解法

1. 載入：fgets 逐行取得資料，紀錄實際行號。
2. 驗證：格式、strtol、範圍、ID 去重與容量全部通過才存入。
3. 輸出：先完成所有驗證，再以 wx 建立報表；逐筆檢查寫入與關閉。

易錯提醒：先輸出報表再驗證剩餘輸入，會留下半份結果。本題先完整載入與驗證，再建立目的檔。

HTML 教學補充界線：本次補充測試具體化了正常、空檔、空白行、ID 重複、超長行與容量超限等情境。所有測試遵守原題無 BOM / NUL 的來源約定；不把範圍外的資料當成已經安全驗證。

15_grade_file.c · 完整程式

```
#include <ctype.h>
#include <errno.h>
#include <stdbool.h>
#include <stdio.h>
#
# 跳至主要內容
#
```

```

static int parse_record(const char *line, int *id, int *score) {
    if (strlen(line) < 6 || line[0] != 'S' || line[4] != ',')
        return 0;
    for (int i = 1; i <= 3; ++i)
        if (!isdigit((unsigned char)line[i])) return 0;
    int number = (line[1] - '0') * 100 +
        (line[2] - '0') * 10 + line[3] - '0';
    char *end;
    errno = 0;
    long value = strtol(line + 5, &end, 10);
    if (end == line + 5 || errno == ERANGE || value < 0 || value > 100)
        return 0;
    while (isspace((unsigned char)*end)) ++end;
    if (*end != '\0') return 0;
    *id = number;
    *score = (int)value;
    return 1;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fputs("usage: grade_file input_csv new_report\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *in = fopen(argv[1], "r");
    if (in == NULL) {
        fputs("Cannot open input\n", stderr);
        return EXIT_FAILURE;
    }
    int ids[CAP], scores[CAP], n = 0, failed = 0;
    bool seen[1000] = {false};
    char line[128];
    size_t line_no = 0;
    while (fgets(line, sizeof line, in) != NULL) {
        ++line_no;
        int id, score;
        if (strchr(line, '\n') == NULL && !feof(in)) {
            fprintf(stderr, "Line %zu: too long\n", line_no);
            failed = 1;
            break;
        }
        if (!parse_record(line, &id, &score)) {
            fprintf(stderr, "Line %zu: invalid record\n", line_no);
            failed = 1;
            break;
        }
        if (seen[id] || n == CAP) {
            fprintf(stderr, "Line %zu: duplicate ID or capacity exceeded\n",
                line_no);
            failed = 1;
            break;
        }
        seen[id] = true;
        ids[n] = id;
        scores[n] = score;
        ++n;
    }
    {
        fputs("it read error\n", stderr);
    }
}

```

[跳至主要內容](#)

```
    if (fclose(in) != 0) failed = 1;
    if (failed) return EXIT_FAILURE;
    if (n == 0) {
        fputs("Empty input\n", stderr);
        return EXIT_FAILURE;
    }
    int sum = 0, pass = 0, low = scores[0], high = scores[0];
    for (int i = 0; i < n; ++i) {
        sum += scores[i];
        if (scores[i] >= 60) ++pass;
        if (scores[i] < low) low = scores[i];
        if (scores[i] > high) high = scores[i];
    }
    FILE *out = fopen(argv[2], "wx");
    if (out == NULL) {
        fputs("Report exists or cannot be created\n", stderr);
        return EXIT_FAILURE;
    }
    if (fprintf(out, "count=%d average=%.2f min=%d max=%d pass=%d\n",
        n, (double)sum / n, low, high, pass) < 0)
        failed = 1;
    for (int i = 0; i < n && !failed; ++i)
        if (fprintf(out, "S%03d,%d,%s\n", ids[i], scores[i],
            scores[i] >= 60 ? "PASS" : "FAIL") < 0)
            failed = 1;
    if (fclose(out) != 0) failed = 1;
    if (failed) {
        fputs("Report write failed\n", stderr);
        if (remove(argv[2]) != 0)
            fputs("Warning: partial report remains\n", stderr);
        return EXIT_FAILURE;
    }
    puts("Report created");
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic 15_grade_file.c -o 15_grade_file
```

測試摘要 (完整輸入與實測見 HTML)

情境	預期返回碼	GCC	Clang
15-01 題目四筆資料	0	PASS	PASS
15-02 最低最高ID	0	PASS	PASS
15-03 分數前後空白與正號	0	PASS	PASS
15-04 末尾不含換行	0	PASS	PASS
15-05 CRLF	0	PASS	PASS
15-06 100筆容量上限	0	PASS	PASS
15-07 126bytes合法列	0	PASS	PASS
15 跳至主要內容	1	PASS	PASS
15-09 重複ID	1	PASS	PASS

情境	預期返回碼	GCC	Clang
15-10 尾端垃圾	1	PASS	PASS
15-11 大於100	1	PASS	PASS
15-12 負分數	1	PASS	PASS
15-13 缺少分數	1	PASS	PASS
15-14 60位數整數	1	PASS	PASS
15-15 空白列	1	PASS	PASS
15-16 ID前空白	1	PASS	PASS
15-17 不接受標題列	1	PASS	PASS
15-18 BOM前綴	1	PASS	PASS
15-19 ID不是三位數	1	PASS	PASS
15-20 127bytes列	1	PASS	PASS
15-21 101筆	1	PASS	PASS
15-22 保護既有報表	1	PASS	PASS
15-23 來源不存在	1	PASS	PASS
15-24 缺少參數	1	PASS	PASS

課堂提問

這個專題在哪三個位置直接使用陣列？

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

教師備忘錄全文 (138 頁)

[跳至主要內容](#)

第 1 頁 | 陣列、檔案 I/O 與錯誤處理

【本頁目標】

陣列、檔案 I/O 與錯誤處理

把三章串成一條可執行、可測試的資料處理流程

【教師講解】

本教材以三份原稿共 245 頁為基礎，重排成陣列、檔案輸入輸出、錯誤處理與整合專題。保留原稿概念與主要案例；新增的規格、測試、圖解及勘誤均有標籤。先用四筆成績的資料流說明：輸入要驗證，陣列要有邊界，統計要符合題意，報表要確認寫入成功。

【課堂提問】

一個程式印出正確平均值，是否就代表整個系統正確？

【參考答案】

未必。還需要確認輸入是否合法、陣列未越界、報表確實寫入且失敗時不毀損既有資料。

【來源與增補】

A 64頁 + B 172頁 + C 9頁

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 2 頁 | 課程地圖 | 從儲存資料到可靠輸出

【本頁目標】

課程地圖 | 從儲存資料到可靠輸出

15 個完整案例，16 支獨立 C 程式；每題先規格、後解法。

【教師講解】

依陣列、I/O、錯誤處理、整合專題順序授課。每個單元以一個具體能力作結，而不是要求學生背函式名稱。案例 10 提供同一題目的逐字元版與區塊版，因此程式檔比題目多一支。可依課程進度拆成多次授課，但完整題目頁應保留在解答之前。

【課堂提問】

案例 10 為何需要兩個版本？

【參考答案】

在同一輸入、輸出和錯誤處理契約下，比較資料搬移單位的差異，避免把最佳化誤當成變更題意。

【來源與增補】

A+B+C

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 3 頁 | 導讀 | 先分清楚三種教材內容

【本頁目標】

導讀 | 先分清楚三種教材內容

主案例採 C17；原稿中的 C99 特性會另外指出。

【教師講解】

提醒學生：檢查 scanf 的回傳值是必要條件，卻不能把不受信任的任意長數字安全轉成 int。為降低初學陣列時的負擔，前七題把「可由型別表示」寫成測試資料前提；到了錯誤處理單元再解除這個限制，改用 fgets 加 strtol。程式碼小片段只用於解釋語法，完整案例另有獨立 .c 檔。

【本頁收斂】

案例固定順序：完整題目 → 核心解法 / 圖解 → 完整程式 → 測試。

【課堂提問】

為何看到 scanf 回傳 1 仍不應宣稱能處理任意外部輸入？

【參考答案】

數值轉換是否可由目標型別表示也是問題；需要可控錯誤回報時，應使用整行讀取加 strtol 與完整驗證。

【來源與增補】

A+B+C | 新增導讀

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 4 頁 | 01 | 陣列：同型別、多筆值、連續元素

【本頁目標】

01 | 陣列：同型別、多筆值、連續元素

一個陣列名稱管理多筆資料；索引決定要存取哪一筆。

【教師講解】

從單一 scalar 變數與 aggregate 變數的差異切入。陣列元素型別相同，元素之間依序配置；圖上的位址假設每個 int 為 4 bytes，只是用來解釋位移，不是所有平台保證。元素 `a[i]` 本身可作為左值讀寫，但整個陣列不是可直接互相指定的普通 scalar。

【本頁收斂】

元素位置： $\text{base} + i \times \text{sizeof}(\text{a}[0])$ ；型別大小以實作為準。

【課堂提問】

`int a[5]` 的最後一個元素是哪個？

【參考答案】

`a[4]`。長度是 5，但合法索引從 0 到 4。

【來源與增補】

A pp.2–7

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 5 頁 | 索引與走訪 | 邊界是程式的責任

【本頁目標】

索引與走訪 | 邊界是程式的責任

C 不保證替你檢查陣列索引；越界存取是未定義行為。

【教師講解】

對照 $i < N$ 與 $i \leq N$ 。示意圖把 $a[N]$ 畫在陣列之外，強調不是某個可依賴的額外元素。越界可能看似正常、破壞其他資料或中止，不應用某次編譯結果當成標準答案。讀入資料前先確認有效筆數，走訪時再維持索引範圍。

【本頁收斂】

合法範圍永遠是 $0 \leq i < N$ ；先驗證，再索引。

【課堂提問】

為何不能把越界程式執行一次後，記住得到的輸出？

【參考答案】

因為未定義行為沒有可攜的結果保證；編譯最佳化、資料配置或平台不同都可能改變行為。

【來源與增補】

A pp.8–11

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 6 頁 | 初始化 | 省略的元素補零，不等於所有陣列自動歸零

【本頁目標】

初始化 | 省略的元素補零，不等於所有陣列自動歸零

區分「有初始列」、「沒有初始列」與儲存期。

【教師講解】

原稿 p.16 提到空初始列合法，與本教材 C17 基準不合；明確標示修正而非悄悄更換。把有初始列但長度不足的零填補，和沒有初始列的區域陣列區分開。不同儲存期有不同規則，所以不要讓學生形成「大括號可有可無」的錯覺。

【表格帶讀】

宣告 / 片段 | 有效內容或意義 | 要注意什麼

`int a[6] = {1, 2};` | 1 2 0 0 0 0 | 有初始列但未指定的元素補零

`int a[6] = {0};` | 全部元素為 0 | C17 可攜的全零寫法

`int a[] = {1, 2, 3};` | 長度由初始列推得為 3 | 宣告時可省略第一維長度

區域 `int a[6];` | 未初始化；不可假設全為 0 | 使用前逐項賦值

`static int a[6];` | 靜態儲存期，初值全為 0 | 規則來自儲存期

`int a[6] = {};` | 本教材 C17 不採用 | 原稿 p.16 的空初始列敘述需勘誤

【課堂提問】

`int a[6]={1,2}` 中，`a[5]` 是不確定值嗎？

【參考答案】

不是，因為有初始列；未指定的其餘元素初始化為零。

【來源與增補】

A pp.14–17 | W1 §6.7.9

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 7 頁 | 指定初始化 | 直接指定索引，建立稀疏表

【本頁目標】

指定初始化 | 直接指定索引，建立稀疏表

C99 designated initializer：位置清楚，比長串 0 更容易維護。

【教師講解】

先展示 `a[15]` 中只有三個位置非零的例子，再展開混合初始化 `c[10]`。學生最常忽略指定索引後，一般值會接續從下一個位置開始。索引必須是整數常數運算式；`const` 查表則用於提醒此資料在程式中不應被修改。

【本頁收斂】

不要用大量看不出位置的 0；在語意重要的位置標明索引。

【課堂提問】

`int c[10]={5,1,9,[4]=3,7,2,[8]=6};` 的 `c[5]` 是多少？

【參考答案】

7。指定 `[4]=3` 後，接著的一般初始值從索引 5 開始。

【來源與增補】

A pp.18–22,46–48

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 8 頁 | sizeof 與 size_t | 知道容量，也要知道適用範圍

【本頁目標】

sizeof 與 size_t | 知道容量，也要知道適用範圍

sizeof 陣列 / sizeof 元素，必須在「真正的陣列」上使用。

【教師講解】

以五元素 int 陣列示範 $20/4=5$ ，但數字 4 是圖解假設。C 使用 size_t，不是原稿截圖中的 C++ std::size_t。陣列作為函式參數時會調整成指標，此時 sizeof a 是指標大小；介面應另外傳入有效長度。size_t 是無號型別，倒數迴圈要避免 $i >= 0$ 永遠成立。

【本頁收斂】

用 size_t 搭配 %zu；長度與容量應成為函式契約的一部分。

【課堂提問】

void f(int a[]) 裡可以用 sizeof a/sizeof a[0] 算出呼叫端長度嗎？

【參考答案】

不行。參數 a 已是指標；必須由呼叫端另外傳入長度。

【來源與增補】

A pp.26–30 | W1 §6.5.3.4

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>（C11 委員會草案，引用與 C17 共通的核心條款）

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 9 頁 | 副作用 | 把索引更新與資料存取拆開

【本頁目標】

副作用 | 把索引更新與資料存取拆開

$a[i++] = 0$ 與 $a[i] = b[i++]$ 不是同一種風險。

【教師講解】

前一式只在該位置使用舊 i ，之後更新 i ，適當邊界下可使用。後一式同時讀 i 作左側索引又在右側修改 i ，C17 下沒有需要的定序，屬未定義行為。不要引導學生猜左邊或右邊先算；用分開敘述或 for 迴圈明確安排順序。

【本頁收斂】

可讀的寫法也比較容易驗證： $a[i] = b[i]$ ；然後再 $++i$ 。

【課堂提問】

能不能用「賦值由右向左」證明 $a[i]=b[i++]$ 正確？

【參考答案】

不能。運算子結合性不等於運算元求值順序；該式在 C17 下不具所需定序。

【來源與增補】

A pp.10–11 | W1 §6.5

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>（C11 委員會草案，引用與 C17 共通的核心條款）

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 10 頁 | 案例 01 | 反向輸出 10 筆量測值

【本頁目標】

案例 01 | 反向輸出 10 筆量測值

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。依序讀入 10 筆整數量測值，保留原本儲存順序，再由最後一筆往前輸出。不得把題目改成排序。每筆介於 $-10000 \sim 10000$ ；缺項、非數字或超出範圍即失敗。入門數值文字約定見導讀頁。

【完整規格】

任務：依序讀入 10 筆整數量測值，保留原本儲存順序，再由最後一筆往前輸出。不得把題目改成排序。

輸入：標準輸入：恰好 10 個整數，以空白或換行分隔。

輸出：由第 10 筆到第 1 筆輸出；數字間一個空白，最後換行。

限制：每筆介於 $-10000 \sim 10000$ ；缺項、非數字或超出範圍即失敗。入門數值文字約定見導讀頁。

範例輸入：

```
34 82 49 102 7 94 23 11 50 31
```

範例輸出：

```
31 50 11 23 94 7 102 49 82 34
```

【課堂提問】

為何輸出迴圈讀 $a[i-1]$ ，而不是 $a[i]$ ？

【參考答案】

i 由元素數量 10 開始；先減 1 才對應最後有效索引 9。 $i > 0$ 也避免 unsigned 倒數下溢。

【來源與增補】

A pp.12–13

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 11 頁 | 案例 01 | 核心解法與圖解

【本頁目標】

案例 01 | 核心解法與圖解

反向輸出 10 筆量測值

【教師講解】

依下列順序帶讀圖解：1. 先以正向迴圈填入 $a[0] \sim a[9]$ 。2. 第二個迴圈從 $i=10$ 遞減，讀取 $a[i-1]$ 。3. 只改變輸出順序，不交換陣列元素。錯誤的起點 $a[10]$ 已越界；最後有效索引是 9。此題不是就地反轉。

【課堂提問】

為何輸出迴圈讀 $a[i-1]$ ，而不是 $a[i]$ ？

【參考答案】

i 由元素數量 10 開始；先減 1 才對應最後有效索引 9。 $i>0$ 也避免 unsigned 倒數下溢。

【來源與增補】

A pp.12–13

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 12 頁 | 案例 01 | 完整程式 1/1

【本頁目標】

案例 01 | 完整程式 1/1

01_reverse.c

【教師講解】

本頁為 01_reverse.c 的第 1/1 段，對應原始檔第 1 ~ 17 行。所有程式頁依序接續，沒有省略程式碼。建立固定長度陣列，逐筆檢查 scanf 回傳值及範圍。讀入全部成功後才由 i=N 向下讀 a[i-1]；這只是反向輸出，沒有修改原陣列次序。

【程式帶讀】

對應 01_reverse.c，原始碼第 1 ~ 17 行。這是完整檔案第 1/1 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
#define N 10

int main(void) {
    int a[N];
    for (int i = 0; i < N; ++i) {
        if (scanf("%d", &a[i]) != 1 ||
            a[i] < -10000 || a[i] > 10000) {
            fputs("Invalid integer input\n", stderr);
            return EXIT_FAILURE;
        }
    }
    for (int i = N; i > 0; --i)
        printf("%d%c", a[i - 1], i == 1 ? '\n' : ' ');
    return EXIT_SUCCESS;
}
```

【課堂提問】

為何輸出迴圈讀 a[i-1]，而不是 a[i]？

【參考答案】

i 由元素數量 10 開始；先減 1 才對應最後有效索引 9。i>0 也避免 unsigned 倒數下溢。

【來源與增補】

A pp.12-13

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼摺 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 13 頁 | 案例 01 | 測試與常見錯誤

【本頁目標】

案例 01 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。錯誤的起點 `a[10]` 已越界；最後有效索引是 9。此題不是就地反轉。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

正常 | 10 筆題目資料 | 31 50 ... 82 34 | 確認前後次序

邊界 | 含負值與 0 | 全部正確反向 | 不是數值排序

缺項 | 1 2 後結束輸入 | 失敗；stderr 訊息 | 不可讀未初始化值

格式錯誤 | 1 2 x | 失敗；stderr 訊息 | 檢查 scanf 回傳

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何輸出迴圈讀 `a[i-1]`，而不是 `a[i]`？

【參考答案】

`i` 由元素數量 10 開始；先減 1 才對應最後有效索引 9。`i>0` 也避免 unsigned 倒數下溢。

【來源與增補】

A pp.12–13

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 14 頁 | 案例 02 | 用布林陣列偵測重複數字

【本頁目標】

案例 02 | 用布林陣列偵測重複數字

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。讀入一個非負整數，從個位數往高位數檢查，找出第一個「再次出現」的數字；完全沒有重複則明確回報。 $0 \leq n \leq 2147483647$ 。 $n=0$ 視為一個數字 0；不保留前導零，負數與錯誤輸入拒絕。

【完整規格】

任務：讀入一個非負整數，從個位數往高位數檢查，找出第一個「再次出現」的數字；完全沒有重複則明確回報。

輸入：標準輸入：一個十進位整數 n 。

輸出：輸出 Repeated digit: d ；若無重複，輸出 No repeated digit。

限制： $0 \leq n \leq 2147483647$ 。 $n=0$ 視為一個數字 0；不保留前導零，負數與錯誤輸入拒絕。

範例輸入：

28212

範例輸出：

Repeated digit: 2

【課堂提問】

為何 $n=0$ 不使用 `while(n>0)` 就結束？

【參考答案】

題目把 0 視為具有一位數的有效輸入；`do...while` 至少執行一次，與題意一致。

【來源與增補】

A pp.23–25

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 15 頁 | 案例 02 | 核心解法與圖解

【本頁目標】

案例 02 | 核心解法與圖解

用布林陣列偵測重複數字

【教師講解】

依下列順序帶讀圖解：1. `seen[10]` 初值全 `false`；索引就是數字 `0~9`。2. `digit=n%10` 取得個位數，先檢查 `seen[digit]`。3. 未出現則設 `true`、`n/=10`；`do...while` 讓 `0` 也被處理一次。不能先把 `seen[digit]` 設為 `true` 再判斷，否則每個數字都會被誤判為重複。

【課堂提問】

為何 `n=0` 不使用 `while(n>0)` 就結束？

【參考答案】

題目把 `0` 視為具有一位數的有效輸入；`do...while` 至少執行一次，與題意一致。

【來源與增補】

A pp.23–25

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 16 頁 | 案例 02 | 完整程式 1/2

【本頁目標】

案例 02 | 完整程式 1/2

02_repeated_digits.c

【教師講解】

本頁為 02_repeated_digits.c 的第 1/2 段，對應原始檔第 1 ~ 14 行。所有程式頁依序接續，沒有省略程式碼。建立長度 10 的 seen 表並全設 false。先驗證非負整數範圍，準備以十進位數字當成狀態表索引。

【程式帶讀】

對應 02_repeated_digits.c，原始碼第 1 ~ 14 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    bool seen[10] = {false};
    long n;
    int repeated = -1;
    if (scanf("%ld", &n) != 1 ||
        n < 0 || n > 2147483647L) {
        fputs("Expected 0..2147483647\n", stderr);
        return EXIT_FAILURE;
    }
    do {
```

【課堂提問】

為何 n=0 不使用 while(n>0) 就結束？

【參考答案】

題目把 0 視為具有一位數的有效輸入；do...while 至少執行一次，與題意一致。

【來源與增補】

A pp.23–25

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 17 頁 | 案例 02 | 完整程式 2/2

【本頁目標】

案例 02 | 完整程式 2/2

02_repeated_digits.c

【教師講解】

本頁為 02_repeated_digits.c 的第 2/2 段，對應原始檔第 15~28 行。所有程式頁依序接續，沒有省略程式碼。用 do-while 讓 0 也能被處理。每次以 %10 取數字，先查表再標記；再次遇見 true 即可回報重複，最後用 /=10 移除末位。

【程式帶讀】

對應 02_repeated_digits.c，原始碼第 15~28 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
int digit = (int)(n % 10);
if (seen[digit]) {
    repeated = digit;
    break;
}
seen[digit] = true;
n /= 10;
} while (n > 0);
if (repeated >= 0)
    printf("Repeated digit: %d\n", repeated);
else
    puts("No repeated digit");
return EXIT_SUCCESS;
}
```

【課堂提問】

為何 n=0 不使用 while(n>0) 就結束？

【參考答案】

題目把 0 視為具有一位數的有效輸入；do...while 至少執行一次，與題意一致。

【來源與增補】

A pp.23–25

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 18 頁 | 案例 02 | 測試與常見錯誤

【本頁目標】

案例 02 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。不能先把 `seen[digit]` 設為 `true` 再判斷，否則每個數字都會被誤判為重複。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

有重複 | 28212 | Repeated digit: 2 | 第 3 次取出的 2 已看過

無重複 | 9357 | No repeated digit | 各索引只設一次

零 | 0 | No repeated digit | 仍應處理一次

零重複 | 100 | Repeated digit: 0 | 兩個 0 需要被辨認

不合法 | -1 | 失敗 | 不可讓 digit 成負索引

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何 `n=0` 不使用 `while(n>0)` 就結束？

【參考答案】

題目把 0 視為具有一位數的有效輸入；`do...while` 至少執行一次，與題意一致。

【來源與增補】

A pp.23–25

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 19 頁 | 案例 03 | 班級成績統計與分布

【本頁目標】

案例 03 | 班級成績統計與分布

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。讀入全班成績，計算總分、平均、最高分、最低分、及格人數，並統計五個成績區間。 $1 \leq n \leq 100$ ； $0 \leq \text{成績} \leq 100$ ；及格為 ≥ 60 。分組：0-59、60-69、70-79、80-89、90-100。

【完整規格】

任務：讀入全班成績，計算總分、平均、最高分、最低分、及格人數，並統計五個成績區間。

輸入：先輸入人數 n ，再輸入 n 個整數成績。

輸出：輸出 count、sum、min、max、average（小數 2 位）、pass 與五個 bins。

限制： $1 \leq n \leq 100$ ； $0 \leq \text{成績} \leq 100$ ；及格為 ≥ 60 。分組：0-59、60-69、70-79、80-89、90-100。

範例輸入：

```
5
59 60 75 90 100
```

範例輸出：

```
count=5 sum=384 min=59 max=100
average=76.80 pass=4
bins=1 1 1 0 2
```

【課堂提問】

為何 `bins[score/10]` 不能直接用在長度 10 的陣列？

【參考答案】

$100/10=10$ ，會存取第 11 格而越界。本題採明確的五區間映射，把 90~100 合併。

【來源與增補】

A pp.8,26-30 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 20 頁 | 案例 03 | 核心解法與圖解

【本頁目標】

案例 03 | 核心解法與圖解

班級成績統計與分布

【教師講解】

依下列順序帶讀圖解：1. 先驗證人數與每筆成績，再開始統計。2. 以第一筆初始化 min/max；用同一趟掃描累加。3. 100 分併入最後一組；平均前轉為 double。sum/n 若先做整數除法再指定給 double，小數已經遺失；需用 (double)sum/n。

【課堂提問】

為何 bins[score/10] 不能直接用在長度 10 的陣列？

【參考答案】

100/10=10，會存取第 11 格而越界。本題採明確的五區間映射，把 90~100 合併。

【來源與增補】

A pp.8,26–30 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 21 頁 | 案例 03 | 完整程式 1/2

【本頁目標】

案例 03 | 完整程式 1/2

03_score_stats.c

【教師講解】

本頁為 03_score_stats.c 的第 1/2 段，對應原始檔第 1 ~ 18 行。所有程式頁依序接續，沒有省略程式碼。配置固定容量 100 的資料陣列與五格 bins。先確認有效筆數與每筆 0 ~ 100 分的範圍；失敗時不讓未完成資料進入統計。

【程式帶讀】

對應 03_score_stats.c，原始碼第 1 ~ 18 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
#define CAP 100

int main(void) {
    int score[CAP], bins[5] = {0};
    int n, sum = 0, pass = 0;
    if (scanf("%d", &n) != 1 || n < 1 || n > CAP) {
        fputs("Expected count 1..100\n", stderr);
        return EXIT_FAILURE;
    }
    for (int i = 0; i < n; ++i) {
        if (scanf("%d", &score[i]) != 1 ||
            score[i] < 0 || score[i] > 100) {
            fputs("Invalid score\n", stderr);
            return EXIT_FAILURE;
        }
    }
}
```

【課堂提問】

為何 bins[score/10] 不能直接用在長度 10 的陣列？

【參考答案】

100/10=10，會存取第 11 格而越界。本題採明確的五區間映射，把 90 ~ 100 合併。

【來源與增補】

A pp.8,26–30 | 新增

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼扣 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 22 頁 | 案例 03 | 完整程式 2/2

【本頁目標】

案例 03 | 完整程式 2/2

03_score_stats.c

【教師講解】

本頁為 03_score_stats.c 的第 2/2 段，對應原始檔第 19~35 行。所有程式頁依序接續，沒有省略程式碼。從第一筆設定最小值與最大值，走訪 n 筆更新總分、及格數與區間。90~100 共用最後一格，避免把 100 分當成索引 5。平均值先轉 double，輸出保留兩位小數。

【程式帶讀】

對應 03_score_stats.c，原始碼第 19~35 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
int low = score[0], high = score[0];
for (int i = 0; i < n; ++i) {
    int s = score[i];
    sum += s;
    if (s < low) low = s;
    if (s > high) high = s;
    if (s >= 60) ++pass;
    int b = s < 60 ? 0 : (s >= 90 ? 4 : s / 10 - 5);
    ++bins[b];
}
printf("count=%d sum=%d min=%d max=%d\n",
       n, sum, low, high);
printf("average=%.2f pass=%d\n", (double)sum / n, pass);
printf("bins=%d %d %d %d %d\n",
       bins[0], bins[1], bins[2], bins[3], bins[4]);
return EXIT_SUCCESS;
}
```

【課堂提問】

為何 bins[score/10] 不能直接用在長度 10 的陣列？

【參考答案】

100/10=10，會存取第 11 格而越界。本題採明確的五區間映射，把 90~100 合併。

【來源與增補】

A pp.8,26–30 | 新增

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼摺 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 23 頁 | 案例 03 | 測試與常見錯誤

【本頁目標】

案例 03 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。sum/n 若先做整數除法再指定給 double，小數已經遺失；需用 (double)sum/n。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

一般 | 59 60 75 90 100 | 平均 76.80；及格 4 | 驗證各組與平均

單筆 | n=1，score=100 | min=max=100 | 初始化不可假設 0

分界 | 0,59,60,69,70,89,100 | bins=2 2 1 1 1 | 確認閉區間

容量 | n=0 或 101 | 失敗 | 先驗證才寫陣列

範圍 | score=101 | 失敗 | 不能當成新分組

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何 bins[score/10] 不能直接用在長度 10 的陣列？

【參考答案】

100/10=10，會存取第 11 格而越界。本題採明確的五區間映射，把 90~100 合併。

【來源與增補】

A pp.8,26–30 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 24 頁 | 案例 04 | 有先後順序的最大增幅

【本頁目標】

案例 04 | 有先後順序的最大增幅

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。量測陣列中的位置代表時間。找出 $\max(a[j]-a[i])$ ，且必須滿足 $i < j$ ；比較雙迴圈與單次掃描兩種解法。 $2 \leq n \leq 100$ ；數值介於 $-10000 \sim 10000$ 。必須選兩個不同時刻；全程下降時答案可能為負。

【完整規格】

任務：量測陣列中的位置代表時間。找出 $\max(a[j]-a[i])$ ，且必須滿足 $i < j$ ；比較雙迴圈與單次掃描兩種解法。

輸入：先輸入 n ，再輸入 n 個整數量測值。

輸出：同一行輸出 brute 與 linear 的結果，兩者必須相同。

限制： $2 \leq n \leq 100$ ；數值介於 $-10000 \sim 10000$ 。必須選兩個不同時刻；全程下降時答案可能為負。

範例輸入：

```
5
10 15 90 200 110
```

範例輸出：

```
brute=190 linear=190
```

【課堂提問】

為何要先算差值，再更新 min_before？

【參考答案】

計算時 min_before 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

【來源與增補】

A pp.59–60 | 新增線性解法

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 25 頁 | 案例 04 | 核心解法與圖解

【本頁目標】

案例 04 | 核心解法與圖解

有先後順序的最大增幅

【教師講解】

依下列順序帶讀圖解：1. 暴力法枚舉所有 $i < j$ ，時間 $O(n^2)$ 。2. 線性法在處理 j 前，保存 $a[0..j-1]$ 的最小值。3. 先用當前值計算增幅，再更新最小值；時間 $O(n)$ 、額外空間 $O(1)$ 。不加順序限制的 max-min 是另一道題。 $\text{best}=0$ 會把全下降資料錯答成 0。

【課堂提問】

為何要先算差值，再更新 min_before ？

【參考答案】

計算時 min_before 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

【來源與增補】

A pp.59–60 | 新增線性解法

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 26 頁 | 案例 04 | 完整程式 1/3

【本頁目標】

案例 04 | 完整程式 1/3

04_ordered_difference.c

【教師講解】

本頁為 04_ordered_difference.c 的第 1/3 段，對應原始檔第 1 ~ 14 行。所有程式頁依序接續，沒有省略程式碼。暴力版只枚舉 $j > i$ 的配對。以第一組合法配對初始化 `best`，而不是以 0 初始化；全部遞減時答案仍可能為負。

【程式帶讀】

對應 04_ordered_difference.c，原始碼第 1 ~ 14 行。這是完整檔案第 1/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
#define CAP 100

static long long brute(const int a[], int n) {
    long long best = (long long)a[1] - a[0];
    for (int i = 0; i < n - 1; ++i)
        for (int j = i + 1; j < n; ++j) {
            long long d = (long long)a[j] - a[i];
            if (d > best) best = d;
        }
    return best;
}
```

【課堂提問】

為何要先算差值，再更新 `min_before`？

【參考答案】

計算時 `min_before` 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

【來源與增補】

A pp.59–60 | 新增線性解法

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 27 頁 | 案例 04 | 完整程式 2/3

【本頁目標】

案例 04 | 完整程式 2/3

04_ordered_difference.c

【教師講解】

本頁為 04_ordered_difference.c 的第 2/3 段，對應原始檔第 15 ~ 28 行。所有程式頁依序接續，沒有省略程式碼。線性版維持處理位置之前的最小值。先計算目前元素可形成的差，再更新前綴最小值。long long 的轉型在相減之前完成。

【程式帶讀】

對應 04_ordered_difference.c，原始碼第 15 ~ 28 行。這是完整檔案第 2/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
static long long linear(const int a[], int n) {
    int min_before = a[0];
    long long best = (long long)a[1] - a[0];
    for (int j = 1; j < n; ++j) {
        long long d = (long long)a[j] - min_before;
        if (d > best) best = d;
        if (a[j] < min_before) min_before = a[j];
    }
    return best;
}
```

```
int main(void) {
    int a[CAP], n;
    if (scanf("%d", &n) != 1 || n < 2 || n > CAP) {
```

【課堂提問】

為何要先算差值，再更新 min_before？

【參考答案】

計算時 min_before 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

【來源與增補】

A pp.59–60 | 新增線性解法

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 28 頁 | 案例 04 | 完整程式 3/3

【本頁目標】

案例 04 | 完整程式 3/3

04_ordered_difference.c

【教師講解】

本頁為 04_ordered_difference.c 的第 3/3 段，對應原始檔第 29~41 行。所有程式頁依序接續，沒有省略程式碼。主程式檢查 $n \geq 2$ 與每筆範圍，再對相同輸入同時呼叫暴力版和線性版。相同答案是基本測試，但還要涵蓋全遞減、相等及邊界案例。

【程式帶讀】

對應 04_ordered_difference.c，原始碼第 29~41 行。這是完整檔案第 3/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
fputs("Expected count 2..100\n", stderr);
return EXIT_FAILURE;
}
for (int i = 0; i < n; ++i) {
    if (scanf("%d", &a[i]) != 1 ||
        a[i] < -10000 || a[i] > 10000) {
        fputs("Invalid sample\n", stderr);
        return EXIT_FAILURE;
    }
}
printf("brute=%lld linear=%lld\n", brute(a, n), linear(a, n));
return EXIT_SUCCESS;
}
```

【課堂提問】

為何要先算差值，再更新 min_before？

【參考答案】

計算時 min_before 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

【來源與增補】

A pp.59–60 | 新增線性解法

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 29 頁 | 案例 04 | 測試與常見錯誤

【本頁目標】

案例 04 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。不加順序限制的 max-min 是另一道題。best=0 會把全下降資料錯答成 0。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

一般 | 10 15 90 200 110 | 190 | 200-10

全下降 | 9 7 3 | -2 | 不能錯設 best=0

相同值 | 4 4 4 | 0 | 仍須兩個位置

兩筆邊界 | -10000 10000 | 20000 | 先轉 long long 再相減

資料不足 | n=1 | 失敗 | 不能讀 a[1]

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何要先算差值，再更新 min_before？

【參考答案】

計算時 min_before 只能來自更早的位置，才能保證 $i < j$ ；先更新會把當前元素當成自己的前值。

【來源與增補】

A pp.59-60 | 新增線性解法

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 30 頁 | 二維陣列 | 表格視覺，row-major 儲存

【本頁目標】

二維陣列 | 表格視覺，row-major 儲存

$a[r][c]$ ：先選第 r 列，再選該列的第 c 欄。

【教師講解】

先用 3×4 表格定位一格，再把整個表格依列攤平成記憶體。 C 的最右側索引變化最快；每一列有 COLS 個元素，所以 $[r][c]$ 的元素偏移為 $r * \text{COLS} + c$ 。此為儲存布局說明，不鼓勵用越過子陣列邊界的指標操作來取代合法索引。

【本頁收斂】

二維索引必須寫 $a[r][c]$ ； $a[r,c]$ 會使用逗號運算子。

【課堂提問】

3×4 陣列的 $a[1][2]$ 距陣列開頭有幾個元素的位置？

【參考答案】

$1 \times 4 + 2 = 6$ ；若圖解假設每個元素 4 bytes，位元組偏移是 24。

【來源與增補】

A pp.36–42

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 31 頁 | 二維初始化與 const 查表

【本頁目標】

二維初始化與 const 查表

保留內層大括號，讓每列長度與用途都看得見。

【教師講解】

將不完整的內層初始列補零與整列省略補零分開示範。單位矩陣以 $r=c$ 決定 1 或 0，也可用指定初始化只填對角線。const 陣列如十六進位或牌碼對照表，文件化不變資料並讓編譯器協助阻止意外修改。

【本頁收斂】

目標：3×3 單位矩陣；核心條件是 $row == col$ ，不是 $row + col$ 。

【課堂提問】

`int m[2][3]={{1},{2,3}}` 的第二列是什麼？

【參考答案】

2、3、0。內層初始列不足時，該列剩餘元素補零。

【來源與增補】

A pp.39–48

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 32 頁 | 案例 05 | 三台設備 × 四時段產量表

【本頁目標】

案例 05 | 三台設備 × 四時段產量表

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。用二維陣列保存三台設備在四個時段的產量，計算每台設備總產量、每時段總產量與全部總量。固定 3 列 × 4 欄；每格 0 ~ 10000。列代表設備，欄代表時段；不可交換索引意義。

【完整規格】

任務：用二維陣列保存三台設備在四個時段的產量，計算每台設備總產量、每時段總產量與全部總量。

輸入：依列輸入 12 個整數：先輸入設備 0 的四時段，再設備 1、設備 2。

輸出：輸出 rows 三筆、cols 四筆，以及 total。

限制：固定 3 列 × 4 欄；每格 0 ~ 10000。列代表設備，欄代表時段；不可交換索引意義。

範例輸入：

10 20 30 40

5 15 25 35

8 12 16 24

範例輸出：

rows=100 80 60

cols=23 47 71 99

total=240

【課堂提問】

為何 rows 和 cols 的陣列長度不同？

【參考答案】

rows 為每台設備保存一個結果，共 3 筆；cols 為每個時段保存一個結果，共 4 筆。

【來源與增補】

A pp.36–46 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 33 頁 | 案例 05 | 核心解法與圖解

【本頁目標】

案例 05 | 核心解法與圖解

三台設備 × 四時段產量表

【教師講解】

依下列順序帶讀圖解：1. 以 `qty[r][c]` 同時表達設備與時段。2. 外層走設備 `r`，內層走時段 `c`。3. 每讀取一格，加入 `rows[r]`、`cols[c]` 與 `total`。`qty[r,c]` 使用的是逗號運算子，不是二維索引；正確寫法必須是 `qty[r][c]`。

【課堂提問】

為何 `rows` 和 `cols` 的陣列長度不同？

【參考答案】

`rows` 為每台設備保存一個結果，共 3 筆；`cols` 為每個時段保存一個結果，共 4 筆。

【來源與增補】

A pp.36–46 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 34 頁 | 案例 05 | 完整程式 1/2

【本頁目標】

案例 05 | 完整程式 1/2

05_matrix_report.c

【教師講解】

本頁為 05_matrix_report.c 的第 1/2 段，對應原始檔第 1 ~ 14 行。所有程式頁依序接續，沒有省略程式碼。以 ROWS、COLS 表達資料語意，準備三列四欄資料、三個列總和、四個欄總和。巢狀迴圈讀入每個合法值。

【程式帶讀】

對應 05_matrix_report.c，原始碼第 1 ~ 14 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
#define ROWS 3
#define COLS 4

int main(void) {
    int qty[ROWS][COLS];
    long rows[ROWS] = {0}, cols[COLS] = {0}, total = 0;
    for (int r = 0; r < ROWS; ++r)
        for (int c = 0; c < COLS; ++c) {
            if (scanf("%d", &qty[r][c]) != 1 ||
                qty[r][c] < 0 || qty[r][c] > 10000) {
                fputs("Invalid quantity\n", stderr);
                return EXIT_FAILURE;
            }
        }
}
```

【課堂提問】

為何 rows 和 cols 的陣列長度不同？

【參考答案】

rows 為每台設備保存一個結果，共 3 筆；cols 為每個時段保存一個結果，共 4 筆。

【來源與增補】

A pp.36–46 | 新增

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 35 頁 | 案例 05 | 完整程式 2/2

【本頁目標】

案例 05 | 完整程式 2/2

05_matrix_report.c

【教師講解】

本頁為 05_matrix_report.c 的第 2/2 段，對應原始檔第 15～28 行。所有程式頁依序接續，沒有省略程式碼。每讀入一個 qty[r][c]，同步更新 rows[r]、cols[c] 及 total。觀察索引的位置和目標總和所代表的維度一致，最後分別輸出。

【程式帶讀】

對應 05_matrix_report.c，原始碼第 15～28 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
    }  
}  
for (int r = 0; r < ROWS; ++r)  
    for (int c = 0; c < COLS; ++c) {  
        rows[r] += qty[r][c];  
        cols[c] += qty[r][c];  
        total += qty[r][c];  
    }  
printf("rows=%ld %ld %ld\n", rows[0], rows[1], rows[2]);  
printf("cols=%ld %ld %ld %ld\n",  
        cols[0], cols[1], cols[2], cols[3]);  
printf("total=%ld\n", total);  
return EXIT_SUCCESS;  
}
```

【課堂提問】

為何 rows 和 cols 的陣列長度不同？

【參考答案】

rows 為每台設備保存一個結果，共 3 筆；cols 為每個時段保存一個結果，共 4 筆。

【來源與增補】

A pp.36–46 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 36 頁 | 案例 05 | 測試與常見錯誤

【本頁目標】

案例 05 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。qty[r,c] 使用的是逗號運算子，不是二維索引；正確寫法必須是 qty[r][c]。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

一般 | 題目 3×4 資料 | 列 100/80/60；總量 240 | 行列語意正確

全部為零 | 12 個 0 | 所有統計皆為 0 | 初值與空產能

單格非零 | qty[1][2]=5，其餘 0 | rows[1]=cols[2]=5 | 定位特定座標

超出限制 | -1 或 10001 | 失敗 | 不能接受非法產量

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何 rows 和 cols 的陣列長度不同？

【參考答案】

rows 為每台設備保存一個結果，共 3 筆；cols 為每個時段保存一個結果，共 4 筆。

【來源與增補】

A pp.36–46 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 37 頁 | 案例 06 | 五種利率的逐年複利表

【本頁目標】

案例 06 | 五種利率的逐年複利表

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。以本金 100 作為數值運算練習，比較起始利率及其後四個整數百分比利率，列出逐年複利後的金額。 $0 \leq \text{rate} \leq 20$ ； $1 \leq \text{years} \leq 30$ 。每年複利一次；本金固定 100。不涉及費用、稅務或投資建議。

【完整規格】

任務：以本金 100 作為數值運算練習，比較起始利率及其後四個整數百分比利率，列出逐年複利後的金額。

輸入：標準輸入：起始年利率 `rate`（整數百分比）與年數 `years`。

輸出：輸出 5 個利率欄，每年一行，金額顯示小數 2 位。

限制： $0 \leq \text{rate} \leq 20$ ； $1 \leq \text{years} \leq 30$ 。每年複利一次；本金固定 100。不涉及費用、稅務或投資建議。

範例輸入：

6 5

範例輸出：

第 5 年：

133.82 140.26 146.93 153.86 161.05

【課堂提問】

為何不需要保存 `years×5` 的整張表？

【參考答案】

下一年只依賴上一年的餘額，已經輸出的歷史資料不再參與計算，因此五個狀態值就足夠。

【來源與增補】

A pp.31–35

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 38 頁 | 案例 06 | 核心解法與圖解

【本頁目標】

案例 06 | 核心解法與圖解

五種利率的逐年複利表

【教師講解】

依下列順序帶讀圖解：1. `value[5]` 分別保存五種情境的「當年餘額」。2. 外層迴圈推進年份；內層更新每個利率。3. `value[i] *= 1.0 + (rate+i)/100.0`；只保存最近一年的狀態。除以 100 會做整數除法；此題必須除以 100.0。顯示四捨五入不代表中間值已被截斷。

【課堂提問】

為何不需要保存 `years×5` 的整張表？

【參考答案】

下一年只依賴上一年的餘額，已經輸出的歷史資料不再參與計算，因此五個狀態值就足夠。

【來源與增補】

A pp.31–35

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 39 頁 | 案例 06 | 完整程式 1/2

【本頁目標】

案例 06 | 完整程式 1/2

06_interest_table.c

【教師講解】

本頁為 06_interest_table.c 的第 1/2 段，對應原始檔第 1 ~ 14 行。所有程式頁依序接續，沒有省略程式碼。確認利率與年數的限制，建立五個初始值同為 100 的 double 元素。先印出五個不同利率的欄位標題。

【程式帶讀】

對應 06_interest_table.c，原始碼第 1 ~ 14 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
#define RATES 5

int main(void) {
    int rate, years;
    double value[RATES];
    if (scanf("%d%d", &rate, &years) != 2 ||
        rate < 0 || rate > 20 || years < 1 || years > 30) {
        fputs("Expected rate 0..20 and years 1..30\n", stderr);
        return EXIT_FAILURE;
    }
    printf("Year");
    for (int i = 0; i < RATES; ++i) {
```

【課堂提問】

為何不需要保存 years×5 的整張表？

【參考答案】

下一年只依賴上一年的餘額，已經輸出的歷史資料不再參與計算，因此五個狀態值就足夠。

【來源與增補】

A pp.31–35

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 40 頁 | 案例 06 | 完整程式 2/2

【本頁目標】

案例 06 | 完整程式 2/2

06_interest_table.c

【教師講解】

本頁為 06_interest_table.c 的第 2/2 段，對應原始檔第 15 ~ 28 行。所有程式頁依序接續，沒有省略程式碼。外層迴圈是年度，內層迴圈是利率欄位。每格使用前一年度同一格的值更新；100.0 確保除法是浮點運算。這是數值迭代教學，不是金融預測。

【程式帶讀】

對應 06_interest_table.c，原始碼第 15 ~ 28 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
    value[i] = 100.0;
    printf(" %7d%%", rate + i);
}
putchar('\n');
for (int year = 1; year <= years; ++year) {
    printf("%4d", year);
    for (int i = 0; i < RATES; ++i) {
        value[i] *= 1.0 + (rate + i) / 100.0;
        printf(" %8.2f", value[i]);
    }
    putchar('\n');
}
return EXIT_SUCCESS;
}
```

【課堂提問】

為何不需要保存 years×5 的整張表？

【參考答案】

下一年只依賴上一年的餘額，已經輸出的歷史資料不再參與計算，因此五個狀態值就足夠。

【來源與增補】

A pp.31–35

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 41 頁 | 案例 06 | 測試與常見錯誤

【本頁目標】

案例 06 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。除以 100 會做整數除法；此題必須除以 100.0。顯示四捨五入不代表中間值已被截斷。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

原稿數值 | rate=6 · years=5 | 第 5 年如右側範例 | 確認遞推

零利率 | rate=0 · years=1 | 第一欄 100.00 | 0% 不增加

最短期間 | years=1 | 只有一列資料 | 迴圈起訖

錯誤年數 | years=0 | 失敗 | 避免無意義輸出

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何不需要保存 years×5 的整張表？

【參考答案】

下一年只依賴上一年的餘額，已經輸出的歷史資料不再參與計算，因此五個狀態值就足夠。

【來源與增補】

A pp.31–35

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 42 頁 | 案例 07 | 不重複發牌與狀態表

【本頁目標】

案例 07 | 不重複發牌與狀態表

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。從 4 種花色、13 種點數的牌組發出 n 張不重複的牌，用二維布林表記錄已發出的牌。 $0 \leq n \leq 52$ 。點數 23456789tjqka；花色 cdhs。同一實作 / 種子可重現；跨編譯器不保證相同牌序。

【完整規格】

任務：從 4 種花色、13 種點數的牌組發出 n 張不重複的牌，用二維布林表記錄已發出的牌。

輸入：標準輸入：張數 n 與 unsigned 整數亂數種子 seed。

輸出：輸出 n 個二字节牌碼，例如 7c；牌碼間以空白分隔。 $n=0$ 只輸出換行。

限制： $0 \leq n \leq 52$ 。點數 23456789tjqka；花色 cdhs。同一實作 / 種子可重現；跨編譯器不保證相同牌序。

範例輸入：

5 1

範例輸出：

5 張合法、互不重複的牌

(精確序列依 C 函式庫而異)

【課堂提問】

發到重複牌時，能否把 dealt 加一？

【參考答案】

不可以。dealt 代表成功輸出的不同牌張數，而不是抽樣嘗試次數。只有確認新牌後才增加。

【來源與增補】

A pp.49–55

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 43 頁 | 案例 07 | 核心解法與圖解

【本頁目標】

案例 07 | 核心解法與圖解

不重複發牌與狀態表

【教師講解】

依下列順序帶讀圖解：1. `rand()%4` 與 `rand()%13` 取得花色、點數。2. `in_hand[suit][rank]` 已為 `true` 就重新抽。3. 新牌才標記、輸出並增加 `dealt`；查表轉成牌碼。`rand()%n` 可能有模數偏差；這是陣列與狀態管理練習，不是公平抽獎或密碼用途的亂數設計。

【課堂提問】

發到重複牌時，能否把 `dealt` 加一？

【參考答案】

不可以。`dealt` 代表成功輸出的不同牌張數，而不是抽樣嘗試次數。只有確認新牌後才增加。

【來源與增補】

A pp.49–55

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 44 頁 | 案例 07 | 完整程式 1/2

【本頁目標】

案例 07 | 完整程式 1/2

07_deal_cards.c

【教師講解】

本頁為 07_deal_cards.c 的第 1/2 段，對應原始檔第 1～15 行。所有程式頁依序接續，沒有省略程式碼。以 4×13 的布林表追蹤每張牌，const 字元表用於把索引轉成點數和花色。輸入張數上限 52，避免要求不可能的手牌。

【程式帶讀】

對應 07_deal_cards.c，原始碼第 1～15 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#define SUITS 4
#define RANKS 13

int main(void) {
    const char ranks[] = "23456789tjqka";
    const char suits[] = "cdhs";
    bool in_hand[SUITS][RANKS] = {{false}};
    int n;
    unsigned seed;
    if (scanf("%d%u", &n, &seed) != 2 || n < 0 || n > 52) {
        fputs("Expected count 0..52 and unsigned seed\n", stderr);
        return EXIT_FAILURE;
    }
```

【課堂提問】

發到重複牌時，能否把 dealt 加一？

【參考答案】

不可以。dealt 代表成功輸出的不同牌張數，而不是抽樣嘗試次數。只有確認新牌後才增加。

【來源與增補】

A pp.49–55

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 45 頁 | 案例 07 | 完整程式 2/2

【本頁目標】

案例 07 | 完整程式 2/2

07_deal_cards.c

【教師講解】

本頁為 07_deal_cards.c 的第 2/2 段，對應原始檔第 16~30 行。所有程式頁依序接續，沒有省略程式碼。初始化亂數種子後反覆挑選 suit 與 rank。只有未出現的牌才標記、輸出並增加已發張數。測試重點是不重複與有效牌值，不是跨平台固定的亂數輸出。

【程式帶讀】

對應 07_deal_cards.c，原始碼第 16~30 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
}
srand(seed);
int dealt = 0;
while (dealt < n) {
    int suit = rand() % SUITS;
    int rank = rand() % RANKS;
    if (in_hand[suit][rank]) continue;
    in_hand[suit][rank] = true;
    printf("%c%c%s", ranks[rank], suits[suit],
        dealt + 1 == n ? "" : " ");
    ++dealt;
}
putchar('\n');
return EXIT_SUCCESS;
}
```

【課堂提問】

發到重複牌時，能否把 dealt 加一？

【參考答案】

不可以。dealt 代表成功輸出的不同牌張數，而不是抽樣嘗試次數。只有確認新牌後才增加。

【來源與增補】

A pp.49–55

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 46 頁 | 案例 07 | 測試與常見錯誤

【本頁目標】

案例 07 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、`stdout`、`stderr` 與檔案內容。`rand()%n` 可能有模數偏差；這是陣列與狀態管理練習，不是公平抽獎或密碼用途的亂數設計。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

不發牌 | `n=0` | 空行 | 不進入發牌迴圈

一般 | `n=5`，`seed=1` | 5 張且不重複 | 驗證性質，不鎖定跨平台牌序

整副牌 | `n=52` | 52 個互異合法牌碼 | 狀態表覆蓋完整牌組

超量 | `n=53` | 立即失敗 | 避免永遠等不到新牌

可重現 | 同一執行檔，同 `seed` | 兩次牌序相同 | 可重現實驗

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

發到重複牌時，能否把 `dealt` 加一？

【參考答案】

不可以。`dealt` 代表成功輸出的不同牌張數，而不是抽樣嘗試次數。只有確認新牌後才增加。

【來源與增補】

A pp.49–55

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 47 頁 | VLA | 執行時決定長度，不是自動成長的容器

【本頁目標】

VLA | 執行時決定長度，不是自動成長的容器

C99 引入；C11 / C17 的支援可依實作而不同。

【教師講解】

保留原教材 VLA 主題，但不把所有 Windows 或所有 C 編譯器都支援當成前提。以固定容量 `cap` 和有效筆數 `n` 說明容量管理是另一個獨立觀念。`n` 驗證需要包含正值和上限；可宣告成功也不等於適合任意大資料。

【課堂提問】

宣告 `int a[n]` 後，執行 `n=1000`，陣列會變成 1000 格嗎？

【參考答案】

不會。該陣列的大小在宣告執行時已決定，不會跟著 `n` 之後的指定而改變。

【來源與增補】

A pp.61–64 | W1 §6.7.6.2

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 48 頁 | 陣列單元收斂 | 先問資料，再選狀態

【本頁目標】

陣列單元收斂 | 先問資料，再選狀態

把每個案例用到的「最小必要狀態」寫清楚。

【教師講解】

這一頁不是再列一次語法，而是讓學生比較不同問題究竟需要保存什麼。布林陣列記錄集成員；一般陣列保存值；前綴最小值則是對歷史資料的摘要。請學生從任務需求說出資料模型，而不是先問要宣告幾維陣列。

【表格帶讀】

任務 | 保存什麼 | 關鍵不變量

反向輸出 | 完整的一維資料 | 讀入 n 筆後才輸出

重複數字 / 發牌 | 是否出現的布林表 | `true` 代表已接受或已觀察

最大有序增幅 | 前綴最小值與最佳答案 | 當前候選只能搭配更早位置

產量表 | 二維原始資料 + 行列統計 | r/c 的語意不交換

複利表 | 每種情境的最近狀態 | 下一年只依賴上一年

【課堂提問】

為何最大增幅的線性演算法不必保存每一組差值？

【參考答案】

因為對固定的 j ，最佳前值必定是之前的最小值；可以用狀態摘要代替所有配對。

【來源與增補】

A pp.1–64 | 重整小結

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 49 頁 | 02 | 輸入 / 輸出：把資料流接到陣列

【本頁目標】

02 | 輸入 / 輸出：把資料流接到陣列

FILE* 是串流介面；陣列是記憶體中的資料模型。

【教師講解】

先把來源、暫存、解析、計算與目的地分開。stdin、檔案與字串可能提供同樣的文字，但需要不同介面接收。成功讀到字元不等於成功解析數值，成功計算也不等於成功寫檔；本節逐一補上這些轉換與回傳值。

【課堂提問】

fgets 成功之後，就能把 line 當成合法整數嗎？

【參考答案】

不能。它只表示讀到字串；還需要解析格式與驗證數值範圍。

【來源與增補】

B pp.5–8 | 新增串接

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 50 頁 | stdin、stdout、stderr | 資料與診斷分流

【本頁目標】

stdin、stdout、stderr | 資料與診斷分流

重導向改變串流的連接，不必改寫程式的計算邏輯。

【教師講解】

沿圖示解釋標準輸入、一般結果與錯誤診斷。指令中的 <、> 和 2> 是 shell 的工作，不是 C 語法。原稿提到 stderr 一定顯示在螢幕，需限縮為「只重導向 stdout 時不受影響」；stderr 本身仍然可以被重導向。範例指令以常見 shell 或 Windows cmd 語法示意。

【本頁收斂】

資料寫 stdout；錯誤寫 stderr；兩者都可能被呼叫環境重導向。

【課堂提問】

prog < in.txt > out.txt 2> err.txt 的錯誤訊息會去哪裡？

【參考答案】

若程式寫到 stderr，診斷會進入 err.txt，而不是螢幕或 out.txt。

【來源與增補】

B pp.3,9–13

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 51 頁 | 跨章橋接 | FILE*、字串與命令列陣列

【本頁目標】

跨章橋接 | FILE*、字串與命令列陣列

只補上本節會用到的介面，不把指標章節整章插進來。

【教師講解】

本節由第 8 章跳到第 22 章，學生可能尚未熟悉字串與指標，因此以三個最小介面建立橋接。清楚區分 FILE*、char[] 與 char*argv[] 的用途。用 argv[1] 指向檔名文字作例，不要求學生自行操作 FILE 內部結構。

【課堂提問】

檔案內容有 1 GB，就需要宣告一個 1 GB 的 FILE 物件嗎？

【參考答案】

不需要。FILE* 管理串流狀態，資料可透過固定大小的緩衝區逐塊處理。

【來源與增補】

B pp.8,38–40,129–135

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 52 頁 | 文字檔與二進位檔 | 相同數值，不同表示法

【本頁目標】

文字檔與二進位檔 | 相同數值，不同表示法

文字保存字元；原生二進位保存實作相關的物件表示。

【教師講解】

以 32767 對照五個十進位字元與二進位表示。圖中位元組順序及大小都明確標示為假設，不能從檔案副檔名判定。

文字模式可能處理換行；通用複製工具使用二進位模式，避免內容在讀寫轉換中改變。原生 int 或 struct 的二進位資料跨平台可能有大小、位元組順序與填補差異。

【本頁收斂】

通用位元組搬移用二進位模式；跨平台交換另定明確檔案格式。

【課堂提問】

副檔名是 .txt，就能在通用複製器中使用 r/w 模式嗎？

【參考答案】

不能只依副檔名假設內容；要保留任意檔案的原始位元組，使用 rb/wb 或建立保護版 wbx。

【來源與增補】

B pp.14–20

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 53 頁 | fopen 模式 | 最重要的是「是否破壞既有資料」

【本頁目標】

fopen 模式 | 最重要的是「是否破壞既有資料」

開啟模式是資料安全的決策，不只是可讀或可寫。

【教師講解】

讓學生先回答資料可不可以被覆寫，再選模式。w 的破壞發生在開檔成功時，不是第一次 fwrite。b 指定二進位模式；x 是排他建立，與 w 搭配使用。程式以相對路徑執行時，位置由目前工作目錄決定；Windows 字串內反斜線需要跳脫。

【表格帶讀】

模式 | 要求檔案存在？ | 允許操作 | 既有內容

r | 是 | 讀 | 保留；位置從開頭

w | 否 | 寫 | 開啟成功時即截斷

a | 否 | 寫（追加） | 保留；寫入追加在尾端

r+ | 是 | 讀 + 寫 | 保留；可定位更新

w+ | 否 | 讀 + 寫 | 開啟成功時即截斷

a+ | 否 | 讀 + 追加寫 | 寫仍追加，初始讀位置依實作

wbx / wx | 必須不存在 | 建立新檔再寫 | 拒絕既有檔案（C11 起）

【課堂提問】

檔案已存在時，fopen(name,"w") 成功但什麼都不寫，原內容還在嗎？

【參考答案】

不在；w 模式成功開啟既有檔案時就會將其截斷。

【來源與增補】

B pp.23–31 | W1 §7.21.5.3 ; W3

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>（C11 委員會草案，引用與 C17 共通的核心條款）

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 54 頁 | 更新串流 | 讀寫切換需要同步點

【本頁目標】

更新串流 | 讀寫切換需要同步點

`r+`、`w+`、`a+` 不是「任何時候隨意混合讀寫」。

【教師講解】

分別画出寫到讀、讀到寫兩個方向。寫到讀可用 `fflush` 或定位函式；讀到寫一般要先定位，只有前次讀取遭遇 EOF 時例外。`fseek` 同時決定接下來要讀寫的實際位置。不要把 `fflush(stdin)` 當作讀到寫的通用解法。

【本頁收斂】

先確定方向，再選同步操作；位置與緩衝區狀態是兩件相關但不同的事。

【課堂提問】

剛 `fscanf` 完資料，直接 `fflush(fp)` 再 `fprintf` 就一定可攜嗎？

【參考答案】

不是。對讀到寫的切換一般需要檔案定位函式；`fflush` 對最近操作為輸入的更新串流不是 ISO C 的可攜解法。

【來源與增補】

B p.31 | W1 §7.21.5.3

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>（C11 委員會草案，引用與 C17 共通的核心條款）

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 55 頁 | 案例 08 | 命令列檔案開啟檢查器

【本頁目標】

案例 08 | 命令列檔案開啟檢查器

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。寫一個命令列工具，接收一個檔名，檢查本次是否能以讀取模式成功開啟並關閉。argc 必須為 2；以本次 fopen 的結果為準。可開啟不代表內容正確，也不保證未來仍可開啟。

【完整規格】

任務：寫一個命令列工具，接收一個檔名，檢查本次是否能以讀取模式成功開啟並關閉。

輸入：命令列：canopen filename；不從鍵盤另外讀取檔名。

輸出：成功：stdout 顯示檔案可開啟。失敗：stderr 顯示原因，回傳 EXIT_FAILURE。

限制：argc 必須為 2；以本次 fopen 的結果為準。可開啟不代表內容正確，也不保證未來仍可開啟。

範例輸入：

canopen sample.txt

範例輸出：

sample.txt can be opened

【課堂提問】

argc 為 2 時，argv[0] 與 argv[1] 分別是什麼？

【參考答案】

argv[0] 是程式名稱字串，argv[1] 是使用者傳入的檔名；argv[argc] 是空指標。

【來源與增補】

B pp.38–42 | C pp.6–9

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 56 頁 | 案例 08 | 核心解法與圖解

【本頁目標】

案例 08 | 核心解法與圖解

命令列檔案開啟檢查器

【教師講解】

依下列順序帶讀圖解：1. 先驗證 argc，再使用 argv[1]。2. fopen 回傳 NULL 就停止；保存 errno 後才輸出訊息。3. 成功開啟也要檢查 fclose；不要把錯誤訊息寫入 stdout。先用「檔案是否存在」判斷再 fopen 仍可能失敗；權限與狀態可能改變，應檢查真正要做的操作。

【課堂提問】

argc 為 2 時，argv[0] 與 argv[1] 分別是什麼？

【參考答案】

argv[0] 是程式名稱字串，argv[1] 是使用者傳入的檔名；argv[argc] 是空指標。

【來源與增補】

B pp.38–42 | C pp.6–9

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 57 頁 | 案例 08 | 完整程式 1/2

【本頁目標】

案例 08 | 完整程式 1/2

08_canopen.c

【教師講解】

本頁為 08_canopen.c 的第 1/2 段，對應原始檔第 1 ~ 14 行。所有程式頁依序接續，沒有省略程式碼。先檢查 argc 才能使用 argv[1]；把 errno 清零後呼叫 fopen，成功與否由 fp 是否為 NULL 決定。

【程式帶讀】

對應 08_canopen.c，原始碼第 1 ~ 14 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fputs("usage: canopen filename\n", stderr);
        return EXIT_FAILURE;
    }
    errno = 0;
    FILE *fp = fopen(argv[1], "r");
    if (fp == NULL) {
        int saved_errno = errno;
        fprintf(stderr, "Cannot open: %s\n", argv[1]);
    }
}
```

【課堂提問】

argc 為 2 時，argv[0] 與 argv[1] 分別是什麼？

【參考答案】

argv[0] 是程式名稱字串，argv[1] 是使用者傳入的檔名；argv[argc] 是空指標。

【來源與增補】

B pp.38–42 | C pp.6–9

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 58 頁 | 案例 08 | 完整程式 2/2

【本頁目標】

案例 08 | 完整程式 2/2

08_canopen.c

【教師講解】

本頁為 08_canopen.c 的第 2/2 段，對應原始檔第 15~27 行。所有程式頁依序接續，沒有省略程式碼。失敗時先保存 `errno`，避免診斷輸出改變它，再視情況以 `perror` 補充原因。成功路徑仍檢查 `fclose`，不把開檔成功當成整段流程已成功。

【程式帶讀】

對應 08_canopen.c，原始碼第 15~27 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
    if (saved_errno != 0) {
        errno = saved_errno;
        perror("fopen");
    }
    return EXIT_FAILURE;
}
if (fclose(fp) != 0) {
    fputs("Close failed\n", stderr);
    return EXIT_FAILURE;
}
printf("%s can be opened\n", argv[1]);
return EXIT_SUCCESS;
}
```

【課堂提問】

`argc` 為 2 時，`argv[0]` 與 `argv[1]` 分別是什麼？

【參考答案】

`argv[0]` 是程式名稱字串，`argv[1]` 是使用者傳入的檔名；`argv[argc]` 是空指標。

【來源與增補】

B pp.38–42 | C pp.6–9

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 59 頁 | 案例 08 | 測試與常見錯誤

【本頁目標】

案例 08 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。先用「檔案是否存在」判斷再 fopen 仍可能失敗；權限與狀態可能改變，應檢查真正要做的操作。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

檔案存在 | sample.txt | 成功並關閉 | 正常流程

不存在 | absent.txt | 失敗；stderr | 不能使用 NULL 串流

漏參數 | canopen | usage 訊息；失敗 | 先驗證 argc

多參數 | canopen a b | usage 訊息；失敗 | 介面規格固定

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

argc 為 2 時，argv[0] 與 argv[1] 分別是什麼？

【參考答案】

argv[0] 是程式名稱字串，argv[1] 是使用者傳入的檔名；argv[argc] 是空指標。

【來源與增補】

B pp.38–42 | C pp.6–9

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 60 頁 | printf 格式 | 把一個規格拆成五個欄位

【本頁目標】

printf 格式 | 把一個規格拆成五個欄位

% [flags] [width] [.precision] [length] conversion

【教師講解】

用原稿的格式文法拆解觀念，再以較容易閱讀的 %+08.2f 示範旗標、寬度和精度。field width 是最小欄寬，不會截斷過長數值；precision 的意義依轉換類型而異。完整規格表移到附錄，主課程先掌握型別匹配與輸出對齊。

【本頁收斂】

格式字串是型別契約；使用者資料要當引數，不要直接當格式字串。

【課堂提問】

printf("%3d",12345) 會只印出 123 嗎？

【參考答案】

不會；欄寬是最小值，完整的 12345 仍會被輸出。

【來源與增補】

B pp.61–85

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 61 頁 | printf 與 scanf | 相似符號，不同參數型別

【本頁目標】

printf 與 scanf | 相似符號，不同參數型別

輸出傳值；輸入多數時候傳入「要被寫入的物件位址」。

【教師講解】

重點對照 double 的輸入 %lf 與輸出 %f。printf 是可變參數函式，float 在傳遞時提升為 double；scanf 寫入物件，float* 和 double* 絕不能互換。陣列名稱用於 %s 時已提供首元素位址，不需要多加不合適的 &。

【表格帶讀】

資料 | printf 片段 | scanf 片段 | 關鍵差異

int | printf("%d", n) | scanf("%d", &n) | scanf 需要 int*

double | printf("%f", x) | scanf("%lf", &x) | 輸入要用 %lf

float | printf("%f", f) | scanf("%f", &f) | 輸出時 float 會提升

size_t | printf("%zu", n) | scanf("%zu", &n) | 不要硬套 %d

字元陣列 | printf("%s", s) | scanf("%15s", s) | s[16] 要保留結尾 \0

物件位址 | printf("%p", (void*)p) | 一般不拿來讀業務資料 | %p 的表示法依實作

【課堂提問】

double x; scanf("%f",&x) 是精度比較少但仍合法嗎？

【參考答案】

不合法；%f 要求 float*，給 double* 是型別不匹配，不能當作正常轉換使用。

【來源與增補】

B pp.71–76,84,86–100

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 62 頁 | 欄寬與精度 | 直接看相同輸入的輸出差異

【本頁目標】

欄寬與精度 | 直接看相同輸入的輸出差異

下表以 · 表示空白；實際輸出沒有圓點。

【教師講解】

用最小欄寬、浮點小數位、有效數字、字串最大長度四組概念避免混淆。原稿 p.81 的 1234.56 用 %.4g 結果有誤，本教材修正為 1235。字串精度以 bytes 計算，不能視為中文字數；多位元組編碼下要特別注意。

【表格帶讀】

格式片段 | 輸入值 | 可見輸出 | 解讀

%8d | 123 |123 | 右靠齊，最小 8 欄

%-8d | 123 | 123..... | 左靠齊

%08d | -123 | -0000123 | 符號後補零

%8.2f | 12.3 | ...12.30 | 小數點後 2 位

%6.4s | "buzzword" | ..buzz | 最多輸出 4 bytes

%.4g | 1234.56 | 1235 | 4 位有效數字；修正原稿 p.81

%.*f | 8, 2, 12.3 | ...12.30 | * 先取寬度，再取精度

【課堂提問】

%.4f 和 %.4g 都是小數點後四位嗎？

【參考答案】

不是。%.4f 是小數點後四位；%.4g 是四位有效數字，並可能採科學記號。

【來源與增補】

B pp.68–83 | p.81 勘誤

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 63 頁 | scanf 配對 | 輸入游標停在哪裡，決定下一次讀到什麼

【本頁目標】

scanf 配對 | 輸入游標停在哪裡，決定下一次讀到什麼

用同一份輸入 12, 34，看空白與常值逗號如何配對。

【教師講解】

先畫出 1、2、空白、逗號、空白、3、4 的位置，再比較四個格式。 %d 會略過前導空白，但格式中的逗號不會自動略過空白。配對失敗時不相符字元留給後續讀取；若迴圈不消耗它，下一次可能在同處再次失敗。

【本頁收斂】

格式中的空白可匹配零個以上輸入空白； %c、 %[、 %n 不自動略過前導空白。

【課堂提問】

輸入是 12, 34， scanf("%d,%d",&a,&b) 回傳多少？

【參考答案】

回傳 1。第一個整數成功後停在空白，格式下一個常值逗號無法立刻配對。

【來源與增補】

B pp.88–109

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 64 頁 | scanf 進階 | 欄寬、抑制指定、scanset 與 %n

【本頁目標】

scanf 進階 | 欄寬、抑制指定、scanset 與 %n

看的是「成功指定幾個物件」，不是消耗了幾段文字。

【教師講解】

先辨識 %s 和 %[會加終止字元、%c 不會。欄寬是輸入項目的最大字元數，不是整行合法性保證。%* 抑制指定，不計入回傳；%n 不消耗輸入，也不計入回傳。避免在格式尾端加不必要的換行，互動輸入可能等待下一個非空白字元。

【表格帶讀】

語法片段 | 輸入 / 容量前提 | 作用與結果

%15s | char s[16] | 最多 15 個非空白字元，再補 \0

%15[^,] | char s[16] | 讀取逗號前內容；至少匹配一個字元

%c / " %c" | char ch | 前者讀下一字元；後者先略過空白

%*d%d | 輸入 12 34 | 略過 12，只指定 34；回傳 1

%*d%d%n | 輸入 10 20 30 | 指定 20；%n 記錄 5；回傳仍 1

%3d | 輸入 12345 | 最多讀 3 字元；不代表完整數字已驗證

【課堂提問】

為何 char s[16] 應使用 %15s 而不是 %16s？

【參考答案】

還需要一格保存字串結尾的空字元；%16s 可能寫入 17 個字元。

【來源與增補】

B pp.95–109

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 65 頁 | 讀取的成功、EOF、錯誤：三種狀態不能混在一起

【本頁目標】

讀取的成功、EOF、錯誤：三種狀態不能混在一起

先嘗試讀取，再依回傳值與串流指標解釋結果。

【教師講解】

強調 `feof` 是之前讀取是否遇到結尾的記錄，不是預測下一次讀取能否成功。`fscanf` 返回成功指定數；若已指定一部分才遇到 EOF，會回傳該數量而不一定是 EOF。`ferror` 與 `feof` 用於解釋串流狀態；配對失敗不一定會設定任何一個指標。

【本頁收斂】

不要把 EOF 當檔案中的某個特定 byte，也不要用它預測下一次讀取。

【課堂提問】

為何 `while(!feof(fp))` 不是正確的讀取迴圈條件？

【參考答案】

因為 EOF 通常要在嘗試讀取之後才被設定；應用讀取函式回傳值控制迴圈，結束後檢查 `ferror`。

【來源與增補】

B pp.88–89,110–116

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 66 頁 | fgets | 容量、換行與字串結尾，都要看得見

【本頁目標】

fgets | 容量、換行與字串結尾，都要看得見

fgets(buf,n,fp) 最多保存 n-1 個讀入字元，並補上 \0。

【教師講解】

以容量 8 的陣列逐格示範讀入 ABCDE 和換行。讀到的換行會保留，puts 又自動加換行，因此直接 puts 原始 line 可能產生多一個空行。長行可能分多次讀到，不是每次成功就代表整行完整。原稿的 gets 是歷史內容；C11 起已移除，本教材不使用。

【本頁收斂】

gets 不列入 C17 解法；puts 自動加換行，fputs 則不會。

【課堂提問】

fgets 成功而字串中沒有換行，一定是完整的一行嗎？

【參考答案】

不一定，可能是最後一行沒有換行，也可能是緩衝區已滿造成截斷；需要依題目長度策略進一步判斷。

【來源與增補】

B pp.129–137 | W1 §7.21.7

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 67 頁 | 字串 I/O | 先接住整行，再解析與格式化

【本頁目標】

字串 I/O | 先接住整行，再解析與格式化

sscanf 解析字串；snprintf 將格式化結果寫進有容量上限的陣列。

【教師講解】

讓學生比較 scanf 直接從串流取資料與 fgets 後再 sscanf 的差別：整行字串可重複檢查，有利錯誤定位。snprintf 的回傳值是沒有長度限制時本來需要寫出的字元數，不包含結束字元；負值代表錯誤，回傳值≥容量表示截斷。成功解析欄位不代表整行與語意都合法，日期案例會補上這一層。

【本頁收斂】

判斷 snprintf：先 $n < 0$ ，再 $(\text{size_t})n \geq \text{sizeof buffer}$ 。

【課堂提問】

`char out[8]; int n=snprintf(out,8,"%s","ABCDEFGH");` n 是 7 嗎？

【參考答案】

不是，n 是原本需要的 9；out 只能保存前 7 個字元加 \0，因此 $n \geq 8$ 表示截斷。

【來源與增補】

B pp.161–171

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 68 頁 | 案例 09 | 日期字串：格式合法不等於日期存在

【本頁目標】

案例 09 | 日期字串：格式合法不等於日期存在

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。讀入一行日期，接受 YYYY-MM-DD 或 YYYY/MM/DD，驗證實際日曆日期後，統一輸出 YYYY-MM-DD。1900 ~ 2099 年；年月日必須分別為 4/2/2 位數；兩個分隔符須相同。不能有多餘字元或空白。輸入為無 NUL 的文字。

【完整規格】

任務：讀入一行日期，接受 YYYY-MM-DD 或 YYYY/MM/DD，驗證實際日曆日期後，統一輸出 YYYY-MM-DD。

輸入：一行 10 個字元的日期；允許行尾 LF、CRLF 或最後一行無換行。

輸出：有效日期：輸出正規化結果；無效：stderr 訊息並失敗。

限制：1900 ~ 2099 年；年月日必須分別為 4/2/2 位數；兩個分隔符須相同。不能有多餘字元或空白。輸入為無 NUL 的文字。

範例輸入：

2024/02/29

範例輸出：

2024-02-29

【課堂提問】

為何本題先檢查數字位置，再呼叫 sscanf？

【參考答案】

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

【來源與增補】

B pp.132–135,161–171 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 69 頁 | 案例 09 | 核心解法與圖解

【本頁目標】

案例 09 | 核心解法與圖解

日期字串：格式合法不等於日期存在

【教師講解】

依下列順序帶讀圖解：1. fgets 取得一整行，只移除行尾 CR/LF。2. 先驗證長度與數字位置，再以 sscanf 拆欄位。3. 檢查月份、閏年與當月天數，最後 snprintf 並檢查截斷。只檢查 sscanf 回傳 3 或 5，只能證明欄位解析成功，不能證明日期在日曆中存在。

【課堂提問】

為何本題先檢查數字位置，再呼叫 sscanf？

【參考答案】

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

【來源與增補】

B pp.132–135,161–171 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 70 頁 | 案例 09 | 完整程式 1/3

【本頁目標】

案例 09 | 完整程式 1/3

09_date_parser.c

【教師講解】

本頁為 09_date_parser.c 的第 1/3 段，對應原始檔第 1 ~ 17 行。所有程式頁依序接續，沒有省略程式碼。以 fgets 取得有限長度文字，僅去掉末端 LF 與 CR；要求日期本體恰為 10 字元，不接受尾端雜字。

【程式帶讀】

對應 09_date_parser.c，原始碼第 1 ~ 17 行。這是完整檔案第 1/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char line[64], sep1, sep2, extra;
    int y, m, d;
    if (fgets(line, sizeof line, stdin) == NULL) {
        fputs("Missing date\n", stderr);
        return EXIT_FAILURE;
    }
    size_t len = strlen(line);
    if (len > 0 && line[len - 1] == '\n') --len;
    if (len > 0 && line[len - 1] == '\r') --len;
    if (len != 10) {
        fputs("Expected YYYY-MM-DD or YYYY/MM/DD\n", stderr);
```

【課堂提問】

為何本題先檢查數字位置，再呼叫 sscanf？

【參考答案】

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

【來源與增補】

B pp.132–135,161–171 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異 [跳至主要內容](#)

第 71 頁 | 案例 09 | 完整程式 2/3

【本頁目標】

案例 09 | 完整程式 2/3

09_date_parser.c

【教師講解】

本頁為 09_date_parser.c 的第 2/3 段，對應原始檔第 18 ~ 34 行。所有程式頁依序接續，沒有省略程式碼。先驗證數字位置及分隔符，再使用有欄寬的 sscanf 取得年月日。年份限制 1900 ~ 2099，兩個分隔符必須相同。

【程式帶讀】

對應 09_date_parser.c，原始碼第 18 ~ 34 行。這是完整檔案第 2/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
    return EXIT_FAILURE;
}
line[len] = '\0';
for (size_t i = 0; i < len; ++i) {
    if (i != 4 && i != 7 &&
        !isdigit((unsigned char)line[i])) {
        fputs("Invalid date digits\n", stderr);
        return EXIT_FAILURE;
    }
}
if (sscanf(line, "%4d%c%2d%c%2d%c",
           &y, &sep1, &m, &sep2, &d, &extra) != 5 ||
    sep1 != sep2 || (sep1 != '-' && sep1 != '/') ||
    y < 1900 || y > 2099 || m < 1 || m > 12) {
    fputs("Invalid date format/range\n", stderr);
    return EXIT_FAILURE;
}
```

【課堂提問】

為何本題先檢查數字位置，再呼叫 sscanf？

【參考答案】

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

【來源與增補】

B pp.132–135,161–171 | 新增

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異 [跳至主要內容](#)

第 72 頁 | 案例 09 | 完整程式 3/3

【本頁目標】

案例 09 | 完整程式 3/3

09_date_parser.c

【教師講解】

本頁為 09_date_parser.c 的第 3/3 段，對應原始檔第 35 ~ 50 行。所有程式頁依序接續，沒有省略程式碼。依月份及閏年規則驗證日期範圍，最後以 `snprintf` 建立 YYYY-MM-DD。檢查回傳長度，完整輸出才回報成功。

【程式帶讀】

對應 09_date_parser.c，原始碼第 35 ~ 50 行。這是完整檔案第 3/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
const int days[] = {31,28,31,30,31,30,31,31,30,31,30,31};
int leap = y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
int limit = days[m - 1] + (m == 2 && leap);
if (d < 1 || d > limit) {
    fputs("Invalid day\n", stderr);
    return EXIT_FAILURE;
}
char out[11];
int written = snprintf(out, sizeof out, "%04d-%02d-%02d", y, m, d);
if (written < 0 || (size_t)written >= sizeof out) {
    fputs("Formatting failed\n", stderr);
    return EXIT_FAILURE;
}
puts(out);
return EXIT_SUCCESS;
}
```

【課堂提問】

為何本題先檢查數字位置，再呼叫 `sscanf`？

【參考答案】

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

【來源與增補】

B pp.132–135,161–171 | 新增

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 73 頁 | 案例 09 | 測試與常見錯誤

【本頁目標】

案例 09 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。只檢查 sscanf 回傳 3 或 5，只能證明欄位解析成功，不能證明日期在日曆中存在。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

閏年 | 2024/02/29 | 2024-02-29 | 可被 4 整除且非百年例外

平年 | 2025-02-29 | 失敗 | 成功解析不代表日期存在

百年例外 | 1900-02-29 | 失敗 | 1900 不是閏年

分隔符混用 | 2024-02/29 | 失敗 | 兩個分隔符需一致

尾端垃圾 | 2024-02-29x | 失敗 | 不可只接受合法前綴

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何本題先檢查數字位置，再呼叫 sscanf？

【參考答案】

先確定欄位恰好是 4/2/2 位數，避免寬度只是上限而非固定長度的誤解，也讓數值一定在可表示範圍內。

【來源與增補】

B pp.132–135,161–171 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 74 頁 | 逐字元 I/O | 用 int 保住 EOF 的區別

【本頁目標】

逐字元 I/O | 用 int 保住 EOF 的區別

fgetc / getchar 回傳值必須先由 int 接收，不要急著縮成 char。

【教師講解】

畫出合法字元值與 EOF 的不同區域。EOF 是負整數常數，不保證是 -1；若先存到 char 可能失去區分能力。fgetc 和 getc 的差異可涉及宏實作，不應直接宣稱其中一個一定比較快；需要效能時量測。ungetc 可退回字元，但不要把 EOF 推回，也不要假設能退回任意多個字元。

【本頁收斂】

正確骨架：while ((ch=fgetc(fp)) != EOF)；離開迴圈後檢查 ferror。

【課堂提問】

二進位檔中遇到 0xFF，可以當成讀到 EOF 嗎？

【參考答案】

不可以。0xFF 是合法檔案 byte；應以 fgetc 的 int 回傳值與 EOF 比較。

【來源與增補】

B pp.117–125

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 75 頁 | fread / fwrite | 「元素數」與「位元組數」要分清楚

【本頁目標】

fread / fwrite | 「元素數」與「位元組數」要分清楚

有效長度 n 是資料的一部分；最後一塊通常沒有填滿緩衝區。

【教師講解】

先介紹參數 ptr、size、count、stream，再以 size=1 的 4097-byte 複製作圖：先讀 4096、再讀 1。回傳是完整元素數；如果 size 不是 1，不能直接當 bytes。二進位 buffer 不必加 \0，不能用 printf("%s") 檢查任意二進位內容。原稿 while(!feof) 的範例在本教材明確改為依 fread 回傳控制。

【本頁收斂】

短讀：再分 EOF / 錯誤；短寫：視為輸出失敗。

【課堂提問】

fread(a,sizeof a[0],10,fp) 回傳 3，代表讀了 3 bytes 嗎？

【參考答案】

不一定；代表成功讀到 3 個完整元素。只有 size=1 時，元素數才等於位元組數。

【來源與增補】

B pp.138–146

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 76 頁 | 案例 10 | 安全複製：逐字元版與區塊版

【本頁目標】

案例 10 | 安全複製：逐字元版與區塊版

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。複製任意文字或二進位檔案，保留所有位元組。實作逐字元與區塊兩版，且不得覆寫任何既有目的檔。目的檔必須不存在；使用 C17 的 `wbx` 建立。檢查讀、寫、關閉錯誤；失敗時嘗試刪除本次建立的不完整檔。

【完整規格】

任務：複製任意文字或二進位檔案，保留所有位元組。實作逐字元與區塊兩版，且不得覆寫任何既有目的檔。

輸入：命令列：`copy_character source new_destination`，或 `copy_block source new_destination`。

輸出：成功：目的檔每個位元組與來源一致，`stdout` 顯示 `Copy complete`。

限制：目的檔必須不存在；使用 C17 的 `wbx` 建立。檢查讀、寫、關閉錯誤；失敗時嘗試刪除本次建立的不完整檔。

範例輸入：

```
copy_block input.bin copy.bin
```

範例輸出：

```
Copy complete
```

```
copy.bin 與 input.bin 位元組完全一致
```

【課堂提問】

區塊版的 `fread(buffer,1,sizeof buffer,in)` 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 77 頁 | 案例 10 | 核心解法與圖解

【本頁目標】

案例 10 | 核心解法與圖解

安全複製：逐字元版與區塊版

【教師講解】

依下列順序帶讀圖解：1. 來源用 `rb`；目的用 `wbx`，避免既有檔案被截斷。2. 逐字元版：`int` 接 `fgetc`；區塊版：以 `n` 傳遞有效長度。3. 結束後查 `ferror` 與 `fclose`；任何失敗都不能宣稱複製完成。若使用 `wb`，開檔當下就可能毀掉原內容。`fwrite` 必須寫本次 `fread` 回傳的 `n`，而不是固定 4096。

【課堂提問】

區塊版的 `fread(buffer,1,sizeof buffer,in)` 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 78 頁 | 案例 10 | 完整程式 1/3

【本頁目標】

案例 10 | 完整程式 1/3

10a_copy_character.c

【教師講解】

本頁為 10a_copy_character.c 的第 1/3 段，對應原始檔第 1 ~ 13 行。所有程式頁依序接續，沒有省略程式碼。驗證來源與目的路徑參數；來源以 rb 開啟，目的以 wbx 建立，已存在的目的檔必須保留不動。

【程式帶讀】

對應 10a_copy_character.c，原始碼第 1 ~ 13 行。這是完整檔案第 1/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {
    if (argc != 3) {
        fputs("usage: copy_character source new_destination\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *in = fopen(argv[1], "rb");
    if (in == NULL) {
        fputs("Cannot open source\n", stderr);
        return EXIT_FAILURE;
    }
}
```

【課堂提問】

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 79 頁 | 案例 10 | 完整程式 2/3

【本頁目標】

案例 10 | 完整程式 2/3

10a_copy_character.c

【教師講解】

本頁為 10a_copy_character.c 的第 2/3 段，對應原始檔第 14~26 行。所有程式頁依序接續，沒有省略程式碼。目的建立失敗就關閉已取得的來源；成功後用 int 接收 fgetc，以 EOF 作讀取結束訊號，逐字元檢查 fputc。

【程式帶讀】

對應 10a_copy_character.c，原始碼第 14~26 行。這是完整檔案第 2/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
FILE *out = fopen(argv[2], "wbx");
if (out == NULL) {
    fputs("Destination exists or cannot be created\n", stderr);
    fclose(in);
    return EXIT_FAILURE;
}
int ch, failed = 0;
while ((ch = fgetc(in)) != EOF) {
    if (fputc(ch, out) == EOF) {
        failed = 1;
        break;
    }
}
```

【課堂提問】

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 80 頁 | 案例 10 | 完整程式 3/3

【本頁目標】

案例 10 | 完整程式 3/3

10a_copy_character.c

【教師講解】

本頁為 10a_copy_character.c 的第 3/3 段，對應原始檔第 27 ~ 38 行。所有程式頁依序接續，沒有省略程式碼。迴圈結束不立即宣告成功；檢查 `ferror` 及兩次 `fclose`。任何失敗只移除本次新建的部分目的檔，並回傳失敗。

【程式帶讀】

對應 10a_copy_character.c，原始碼第 27 ~ 38 行。這是完整檔案第 3/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
if (ferror(in)) failed = 1;
if (fclose(in) != 0) failed = 1;
if (fclose(out) != 0) failed = 1;
if (failed) {
    fputs("Copy failed; removing partial output\n", stderr);
    if (remove(argv[2]) != 0)
        fputs("Warning: partial output remains\n", stderr);
    return EXIT_FAILURE;
}
puts("Copy complete");
return EXIT_SUCCESS;
}
```

【課堂提問】

區塊版的 `fread(buffer,1,sizeof buffer,in)` 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 81 頁 | 案例 10 | 完整程式 1/3

【本頁目標】

案例 10 | 完整程式 1/3

10b_copy_block.c

【教師講解】

本頁為 10b_copy_block.c 的第 1/3 段，對應原始檔第 1 ~ 14 行。所有程式頁依序接續，沒有省略程式碼。建立與逐字元版相同的命令列與開檔契約。先 rb 開來源，再以 wbx 拒絕覆寫目的檔。

【程式帶讀】

對應 10b_copy_block.c，原始碼第 1 ~ 14 行。這是完整檔案第 1/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fputs("usage: copy_block source new_destination\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *in = fopen(argv[1], "rb");
    if (in == NULL) {
        fputs("Cannot open source\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *out = fopen(argv[2], "wbx");
```

【課堂提問】

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 82 頁 | 案例 10 | 完整程式 2/3

【本頁目標】

案例 10 | 完整程式 2/3

10b_copy_block.c

【教師講解】

本頁為 10b_copy_block.c 的第 2/3 段，對應原始檔第 15～27 行。所有程式頁依序接續，沒有省略程式碼。使用 unsigned char buffer[4096] 搬移原始 bytes。fread 的 size 為 1，回傳值因此直接是有效 byte 數 n；fwrite 必須使用 n。

【程式帶讀】

對應 10b_copy_block.c，原始碼第 15～27 行。這是完整檔案第 2/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
if (out == NULL) {
    fputs("Destination exists or cannot be created\n", stderr);
    fclose(in);
    return EXIT_FAILURE;
}
unsigned char buffer[4096];
size_t n;
int failed = 0;
while ((n = fread(buffer, 1, sizeof buffer, in)) > 0) {
    if (fwrite(buffer, 1, n, out) != n) {
        failed = 1;
        break;
    }
}
```

【課堂提問】

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 83 頁 | 案例 10 | 完整程式 3/3

【本頁目標】

案例 10 | 完整程式 3/3

10b_copy_block.c

【教師講解】

本頁為 10b_copy_block.c 的第 3/3 段，對應原始檔第 28~40 行。所有程式頁依序接續，沒有省略程式碼。短讀後檢查來源錯誤，並檢查輸入與輸出的關閉結果。失敗時清理本次新建的部分檔案；成功才印出 Copy complete。

【程式帶讀】

對應 10b_copy_block.c，原始碼第 28~40 行。這是完整檔案第 3/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
}
if (ferror(in)) failed = 1;
if (fclose(in) != 0) failed = 1;
if (fclose(out) != 0) failed = 1;
if (failed) {
    fputs("Copy failed; removing partial output\n", stderr);
    if (remove(argv[2]) != 0)
        fputs("Warning: partial output remains\n", stderr);
    return EXIT_FAILURE;
}
puts("Copy complete");
return EXIT_SUCCESS;
}
```

【課堂提問】

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 84 頁 | 案例 10 | 測試與常見錯誤

【本頁目標】

案例 10 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。若使用 wb，開檔當下就可能毀掉原內容。fwrite 必須寫本次 fread 回傳的 n，而不是固定 4096。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

二進位 | 含 00、1A、FF 的檔案 | 位元組完全一致 | 不能當成 C 字串

空檔 | 0 bytes | 建立 0-byte 目的檔 | EOF 不是錯誤

區塊邊界 | 4096 / 4097 bytes | 無遺失、無額外資料 | 最後一塊只寫 n bytes

目的已存在 | 既有檔內容 KEEP | 失敗；KEEP 不變 | x 模式拒絕覆寫

同一路徑 | 來源=目的 | 失敗；來源不變 | 避免先截斷來源

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

區塊版的 fread(buffer,1,sizeof buffer,in) 回傳的是什麼？

【參考答案】

回傳完整元素數；因每個元素大小設成 1 byte，所以這個案例的回傳值也就是有效位元組數。

【來源與增補】

B pp.117–128,138–146 | 新增防護

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 85 頁 | 緩衝區 | 函式返回，不代表資料已永久存入磁碟

【本頁目標】

緩衝區 | 函式返回，不代表資料已永久存入磁碟

程式 → stdio buffer → 主機環境 / 作業系統 → 儲存裝置

【教師講解】

沿四層流程說明 buffering 減少細碎的資料搬移。fflush 把 stdio 緩衝區送到主機環境，不是斷電持久化保證。

fclose 也會處理尚未寫出的資料，錯誤可能到此時才被發現。將性能與可靠性分開說明，不能因每個 printf 看似成功就省略關閉檢查。

【課堂提問】

fflush(fp)==0 是否保證突然斷電後檔案仍完整？

【參考答案】

不是。它是標準 I/O 層的刷新成功，不是所有儲存層的持久化或交易保證。

【來源與增補】

B pp.49–57

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 86 頁 | setvbuf 與 fflush | 時間點與生命週期比呼叫名稱重要

【本頁目標】

setvbuf 與 fflush | 時間點與生命週期比呼叫名稱重要

先設緩衝，再做其他串流操作；使用者提供的 buffer 不能提早失效。

【教師講解】

原稿 p.57 在已經輸出後才 setvbuf，與前頁規則矛盾，本教材明確修正。若 buffer 是區域陣列，其生命週期必須涵蓋串流使用期間。setbuf 沒有回傳狀態，不代表 C17 已移除它。清掉不合法輸入應讀走剩餘字元或採整行解析，不使用 fflush(stdin)。

【表格帶讀】

工具 / 模式 | 用途 | 使用條件與注意

_IOFBF | 全緩衝 | 適合區塊搬移；由大小等條件觸發刷新

_IOLBF | 行緩衝 | 與換行及互動輸入等條件相關

_IONBF | 無緩衝 | 不是所有輸出都需要如此設定

setvbuf(fp,buf,mode,n) | 設定模式 / 大小 / buffer | 開啟後、其他 I/O 前呼叫；檢查非零失敗

fflush(fp) | 刷新適用的輸出串流 | 成功 0；失敗 EOF；不是清除輸入

setbuf / fflush(stdin) | 舊介面 / 常見誤用 | setbuf 仍在 C17；fflush(stdin) 不可攜

【課堂提問】

可以先 printf 提示，再 setvbuf(stdout,...) 嗎？

【參考答案】

可攜寫法應先設定緩衝，再對該串流進行其他 I/O；不要照抄原稿順序錯誤的示例。

【來源與增補】

B pp.51–57 | W1 §7.21.5

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 87 頁 | 檔案定位 | 把記錄索引換成 byte offset

【本頁目標】

檔案定位 | 把記錄索引換成 byte offset

固定長度二進位記錄： $\text{offset} = \text{index} \times \text{record_size}$ 。

【教師講解】

以五筆 int 記錄示範索引 2 的位置，但數字 4 bytes 只作圖解假設。fseek 傳入 long 偏移及 SEEK_SET/CUR/END 原點，回傳 0 代表定位操作成功；定位到某處不代表那裡有完整記錄，仍要檢查 fread。ftell 失敗可回傳 -1L。更新串流還須符合讀寫切換規則。

【本頁收斂】

位置是下一次讀寫的起點；讀完一筆後，位置已前進。

【課堂提問】

fseek 成功能證明第 100 筆記錄存在嗎？

【參考答案】

不能。可能只是成功調整位置；要讀到指定數量的完整記錄，仍需檢查 fread 的回傳值。

【來源與增補】

B pp.148–156

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 88 頁 | 定位的邊界 | 文字串流不適合任意 byte 算術

【本頁目標】

定位的邊界 | 文字串流不適合任意 byte 算術

把「回到曾經的位置」與「計算位元組偏移」分開。

【教師講解】

保留原稿的五個定位函式，但修正容易被理解成「fpos_t 永遠支援任意大檔」的說法。文字模式可能存在編碼與換行轉換，ftell 值適合當作可恢復的位置標記，不一定是可自行運算的檔案 byte 數。clearerr 只清狀態，不能被當作任何錯誤的修復。

【表格帶讀】

介面 / 情境 | 可用方式 | 不可過度推論

二進位 fseek / ftell | 以 byte 計數，注意 long 可表示範圍 | SEEK_END 並非每個二進位串流都必須支援

文字 fseek | offset=0，或用先前 ftell 值配 SEEK_SET | 不要對文字 ftell 結果任意加減

rewind(fp) | 回到開頭並清除 EOF / 錯誤指標 | 無回傳值，難以直接檢查定位失敗

fgetpos / fsetpos | 以 fpos_t 保存與還原位置 | fpos_t 未必是整數，也非無限檔案大小保證

clearerr(fp) | 清除 EOF 與 error 指標 | 不移動位置，也不修復故障原因

【課堂提問】

clearerr(fp) 後，檔案位置會回到開頭嗎？

【參考答案】

不會。它只清除 EOF 和錯誤指標；位置需要 rewind、fseek 或其他定位函式處理。

【來源與增補】

B pp.112–114,152–159 | 勘誤 / 補充

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 89 頁 | 案例 11 | 直接更新庫存檔的第 3 筆

【本頁目標】

案例 11 | 直接更新庫存檔的第 3 筆

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。建立五筆整數庫存的二進位檔，讀出索引 2 的舊值，再直接改成 99，最後重讀整個檔案確認結果。目的檔必須不存在；第 3 筆 = 索引 2。原生 int 格式僅限相同實作讀寫；這是定位練習，不提供交易式還原。

【完整規格】

任務：建立五筆整數庫存的二進位檔，讀出索引 2 的舊值，再直接改成 99，最後重讀整個檔案確認結果。

輸入：命令列：stock_update new_binary_file。初始資料固定為 10、20、35、40、50。

輸出：顯示 old=35 new=99，及更新後的五筆整數。

限制：目的檔必須不存在；第 3 筆 = 索引 2。原生 int 格式僅限相同實作讀寫；這是定位練習，不提供交易式還原。

範例輸入：

```
stock_update stock.bin
```

範例輸出：

```
old=35 new=99
```

```
10 20 99 40 50
```

【課堂提問】

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

【參考答案】

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

【來源與增補】

B pp.138–160 | 原庫存案例改編

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 90 頁 | 案例 11 | 核心解法與圖解

【本頁目標】

案例 11 | 核心解法與圖解

直接更新庫存檔的第 3 筆

【教師講解】

依下列順序帶讀圖解：1. 先 wbx 寫入初始陣列，關閉後以 rb+ 重開。2. 位置 = 2×sizeof(int)；先 fseek 再讀出舊值。3. 讀→寫前重新定位；寫→讀前再定位到檔首。第 k 筆以 0-based 索引 k 計算 offset；更換平台不能假設 int 永遠是 4 bytes，也不能忽略位元組順序。

【課堂提問】

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

【參考答案】

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

【來源與增補】

B pp.138–160 | 原庫存案例改編

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 91 頁 | 案例 11 | 完整程式 1/3

【本頁目標】

案例 11 | 完整程式 1/3

11_stock_update.c

【教師講解】

本頁為 11_stock_update.c 的第 1/3 段，對應原始檔第 1 ~ 15 行。所有程式頁依序接續，沒有省略程式碼。建立含五筆原生 int 的新二進位檔。使用 wbx 確保不覆寫既有檔案，並檢查最初 fwrite 與 fclose。

【程式帶讀】

對應 11_stock_update.c，原始碼第 1 ~ 15 行。這是完整檔案第 1/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {
    int stock[5] = {10, 20, 35, 40, 50};
    int updated = 99, previous;
    int failed = 0;
    if (argc != 2) {
        fputs("usage: stock_update new_binary_file\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *fp = fopen(argv[1], "wbx");
    if (fp == NULL) {
        fputs("Cannot create new stock file\n", stderr);
        return EXIT_FAILURE;
    }
}
```

【課堂提問】

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

【參考答案】

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

【來源與增補】

B pp.138–160 | 原庫存案例改編

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 92 頁 | 案例 11 | 完整程式 2/3

【本頁目標】

案例 11 | 完整程式 2/3

11_stock_update.c

【教師講解】

本頁為 11_stock_update.c 的第 2/3 段，對應原始檔第 16~30 行。所有程式頁依序接續，沒有省略程式碼。以 rb+ 重新開啟，計算索引 2 的 byte 位移。先定位再讀舊值；讀取會前進，因此改寫前必須再次定位。

【程式帶讀】

對應 11_stock_update.c，原始碼第 16~30 行。這是完整檔案第 2/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
}
if (fwrite(stock, sizeof stock[0], 5, fp) != 5) failed = 1;
if (fclose(fp) != 0) failed = 1;
if (failed) {
    fputs("Initial write failed\n", stderr);
    return EXIT_FAILURE;
}
fp = fopen(argv[1], "rb+");
if (fp == NULL) {
    fputs("Cannot reopen stock file\n", stderr);
    return EXIT_FAILURE;
}
long offset = 2L * (long)sizeof stock[0];
if (fseek(fp, offset, SEEK_SET) != 0 ||
    fread(&previous, sizeof previous, 1, fp) != 1 ||
```

【課堂提問】

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

【參考答案】

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

【來源與增補】

B pp.138–160 | 原庫存案例改編

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 93 頁 | 案例 11 | 完整程式 3/3

【本頁目標】

案例 11 | 完整程式 3/3

11_stock_update.c

【教師講解】

本頁為 11_stock_update.c 的第 3/3 段，對應原始檔第 31 ~ 44 行。所有程式頁依序接續，沒有省略程式碼。寫入新值後定位回開頭以便重新讀取。檢查讀寫與關閉結果，列出舊值、新值與完整陣列。此格式只適用同一實作，不提供跨平台或交易保證。

【程式帶讀】

對應 11_stock_update.c，原始碼第 31 ~ 44 行。這是完整檔案第 3/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
fseek(fp, offset, SEEK_SET) != 0 ||
fwrite(&updated, sizeof updated, 1, fp) != 1 ||
fseek(fp, 0L, SEEK_SET) != 0 ||
fread(stock, sizeof stock[0], 5, fp) != 5) {
    fputs("Record update/read failed\n", stderr);
    failed = 1;
}
if (fclose(fp) != 0) failed = 1;
if (failed) return EXIT_FAILURE;
printf("old=%d new=%d\n", previous, stock[2]);
for (int i = 0; i < 5; ++i)
    printf("%d%c", stock[i], i == 4 ? '\n' : ' ');
return EXIT_SUCCESS;
}
```

【課堂提問】

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

【參考答案】

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

【來源與增補】

B pp.138–160 | 原庫存案例改編

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 94 頁 | 案例 11 | 測試與常見錯誤

【本頁目標】

案例 11 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。第 k 筆以 0-based 索引 k 計算 offset；更換平台不能假設 int 永遠是 4 bytes，也不能忽略位元組順序。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

一般 | 新檔 stock.bin | 35→99，其餘不變 | 精準更新一筆

檔案內容 | 5 個原生 int | 10 20 99 40 50 | 整檔驗證

既有檔案 | 再次執行同檔名 | 失敗；不重建 | 避免覆寫

漏參數 | stock_update | usage；失敗 | 命令列契約

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

讀完索引 2 後，為什麼不能直接 fwrite 覆寫該筆？

【參考答案】

讀取後檔案位置已前進到下一筆，且更新串流的讀→寫需要定位；必須重新 fseek 回索引 2 的起點。

【來源與增補】

B pp.138–160 | 原庫存案例改編

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 95 頁 | 其他檔案工具 | 名稱、串流與暫存資料各有介面

【本頁目標】

其他檔案工具 | 名稱、串流與暫存資料各有介面

保留原稿工具，但把限制與風險一起教。

【教師講解】

這些工具容易因為函式名稱直觀就被誤用。tmpnam 並沒有保留那個名稱，取得名字到開檔之間可能改變；因此不可教成已保證安全建立。remove 和 rename 的成功與否仍依回傳值確認。寬字元與可變參數的延伸保留索引，不在此處跳過必要前置概念。

【表格帶讀】

介面 | 處理對象 | 正確期待

freopen | 既有串流 | 可重新關聯 stdin/stdout；仍需檢查 NULL

tmpfile | 暫存串流 | 以更新二進位模式建立；關閉或正常結束時移除

tmpnam | 候選檔名 | 只產生名稱、不建立檔案；有先後競爭風險

remove | 檔名 | 成功 0；失敗非零；不保證可移除使用中的檔

rename | 舊檔名與新檔名 | 成功 0；新名稱已存在時效果依實作

v*printf / 寬字元 I/O | va_list / wchar_t | 延伸介面；與本教材 byte I/O 核心分層學習

【課堂提問】

tmpnam 回傳一個名字，代表檔案已建立好了嗎？

【參考答案】

沒有。它只產生名稱；需要另行建立，而兩個動作之間存在狀態改變的可能。

【來源與增補】

B pp.35–37,43–48,58–60,65,162

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 96 頁 | I/O 選擇表 | 依資料單位選函式，再檢查回傳

【本頁目標】

I/O 選擇表 | 依資料單位選函式，再檢查回傳

同一問題可用不同 I/O 單位，但成功條件不能含糊。

【教師講解】

讓學生從要處理的是欄位、字元、行還是區塊開始選擇，而不是慣性使用 `scanf`。最後一列是本教材新增的強健輸入流程，將輸入取得與數字轉換分離。任何一欄都應列出失敗時如何停止或恢復，下一單元會把這些策略統一。

【表格帶讀】

需要處理的單位 | 讀取 | 輸出 | 成功判斷重點

格式化欄位 | `scanf` / `fscanf` | `printf` / `fprintf` | 指定數是否足夠；輸出是否負值

單一字元 | `getchar` / `fgetc` | `putchar` / `fputc` | `int` 與 `EOF`；錯誤指標

文字行 / 字串 | `fgets` | `puts` / `fputs` | `NULL`、截斷、換行、終止字元

二進位區塊 | `fread` | `fwrite` | 完整元素數；最後一塊長度

記憶體中的字串 | `sscanf` | `snprintf` | 完整解析；輸出截斷

任意數字文字 | `fgets` + `strtol` | 驗證後再格式化 | `end`、`ERANGE`、業務範圍

【課堂提問】

大量二進位資料要用 `fgets` 配 `printf("%s")` 嗎？

【參考答案】

不應；二進位可能包含零 `byte`，也不具字串結尾保證。應使用 `fread/fwrite` 和有效長度。

【來源與增補】

B pp.61–172 | 新增彙整

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 97 頁 | 03 | 錯誤處理：分辨錯誤來源，才能選對處置

【本頁目標】

03 | 錯誤處理：分辨錯誤來源，才能選對處置

輸入錯誤、外部資源失敗、內部邏輯違反，是不同問題。

【教師講解】

以三類錯誤建立分類。使用者輸入不合法應能用一般控制流程拒絕；檔案失敗需要診斷並清理；內部不變量違反可由 `assert` 協助找出開發缺陷。不是所有錯誤都應重試，也不是全部都應 `abort`。成功訊息與返回狀態要和真正完成的副作用一致。

【課堂提問】

使用者輸入 101 分，適合用 `assert` 中止程式嗎？

【參考答案】

不適合以 `assert` 作主要輸入驗證。這是可預期的外部資料錯誤，應回報原因並以正常流程處理。

【來源與增補】

C pp.1–9 | B pp.110–116

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 98 頁 | 回傳值、ferror、errno | 三個不同層次的訊息

【本頁目標】

回傳值、ferror、errno | 三個不同層次的訊息

先看函式的成功 / 失敗契約，再決定還要查看哪個診斷來源。

【教師講解】

errno 並不是每次呼叫都會自動清除的萬用旗標。先用 fopen 是否 NULL、fread 是否足量等回傳值判定發生什麼事，再依該函式規格查診斷。數字轉換與數學函式的 errno 檢查方式也不完全相同；保存後才做其他操作，避免覆寫。

【本頁收斂】

先判斷操作結果；需要診斷時，立即保存 errno。

【課堂提問】

errno!=0 可以單獨證明剛才的 fopen 失敗嗎？

【參考答案】

不可以；可能是先前留下的值。fopen 成功與否看回傳指標，errno 只依平台與函式規格補充診斷。

【來源與增補】

C pp.6–9 | B pp.110–114

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 99 頁 | perror 與 strerror | 回報原因，不是自動恢復

【本頁目標】

perror 與 strerror | 回報原因，不是自動恢復

錯誤描述要包含操作、檔名或行號，不能只丟一句 Error。

【教師講解】

指出原稿 POSIX 標示的意義，不把平台擴充碼一律當成 ISO C 規定。perror 直接輸出，strerror 提供文字給呼叫者自行組合；兩者都不會修正檔案或重新執行操作。最好同時提供程式自己的上下文，而不是只依賴系統字串。

【表格帶讀】

工具 / 資訊 | 來源與結果 | 使用提醒

perror("open") | 依目前 errno 寫入 stderr | 先確認 errno 是相關失敗的診斷

strerror(saved_errno) | 回傳對應的說明文字 | 文字與語言依實作，不做固定字串比對

EDOM / ERANGE / EILSEQ | ISO C 提供的錯誤巨集 | 定義域、範圍 / 極點、非法位元組序列

ENOENT / EACCES 等 | POSIX / 平台常見錯誤碼 | 不可當成所有 ISO C 實作必備

檔名 / 行號 / 操作 | 程式自己掌握的上下文 | 讓使用者能找到哪個資料或步驟失敗

【課堂提問】

把 strerror 的英文句子寫死在跨平台測試裡，穩定嗎？

【參考答案】

不穩定。應檢查回傳狀態或錯誤碼的意義，說明文字可能因平台或語系而不同。

【來源與增補】

C pp.2,6–9 | W1 §7.5,7.21.10,7.24.6

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 100 頁 | 案例 12 | 嚴格讀取一個 0 ~ 100 分的整數

【本頁目標】

案例 12 | 嚴格讀取一個 0 ~ 100 分的整數

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。讀入一行成績，不只判斷是否以數字開頭，還要拒絕超大整數、尾端垃圾、空白行與超長輸入。數值 0 ~ 100；單行最多 126 bytes (不含 LF)；不接受小數或第二個值。最末行可不含換行。

【完整規格】

任務：讀入一行成績，不只判斷是否以數字開頭，還要拒絕超大整數、尾端垃圾、空白行與超長輸入。

輸入：標準輸入的一行無 NUL 文字；允許數字前後空白及正號。

輸出：有效：score=value。無效：stderr 回報 No number、Trailing text、Out of range 等。

限制：數值 0 ~ 100；單行最多 126 bytes (不含 LF)；不接受小數或第二個值。最末行可不含換行。

範例輸入：

+86

範例輸出：

score=86

【課堂提問】

為何 *out=(int)value 要放在所有檢查之後？

【參考答案】

只有值已落在 0 ~ 100，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

【來源與增補】

B pp.103–104,132–135 | C pp.6–9 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 101 頁 | 案例 12 | 核心解法與圖解

【本頁目標】

案例 12 | 核心解法與圖解

嚴格讀取一個 0 ~ 100 分的整數

【教師講解】

依下列順序帶讀圖解：1. fgets 限制讀取容量；過長輸入整行拒絕。2. errno=0 後 strtol，保留 end 指標。3. 依序檢查沒讀到數字、ERANGE / 範圍、尾端非空白；成功才設定輸出值。atoi 無法清楚區分「合法 0」與轉換失敗；只看 errno 也無法偵測 86abc。

【課堂提問】

為何 *out=(int)value 要放在所有檢查之後？

【參考答案】

只有值已落在 0 ~ 100，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

【來源與增補】

B pp.103–104,132–135 | C pp.6–9 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 102 頁 | 案例 12 | 完整程式 1/3

【本頁目標】

案例 12 | 完整程式 1/3

12_strict_score.c

【教師講解】

本頁為 12_strict_score.c 的第 1/3 段，對應原始檔第 1～15 行。所有程式頁依序接續，沒有省略程式碼。狀態 enum 把成功、缺少數字、尾端雜字及範圍錯誤分開。strtol 前將 errno 清零，轉換後用 end 與 ERANGE 判斷。

【程式帶讀】

對應 12_strict_score.c，原始碼第 1～15 行。這是完整檔案第 1/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum Status { OK, NO_NUMBER, TRAILING, OUT_OF_RANGE };

static enum Status parse_score(const char *text, int *out) {
    char *end;
    errno = 0;
    long value = strtol(text, &end, 10);
    if (end == text) return NO_NUMBER;
    if (errno == ERANGE || value < 0 || value > 100)
        return OUT_OF_RANGE;
```

【課堂提問】

為何 *out=(int)value 要放在所有檢查之後？

【參考答案】

只有值已落在 0～100，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

【來源與增補】

B pp.103–104,132–135 | C pp.6–9 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 103 頁 | 案例 12 | 完整程式 2/3

【本頁目標】

案例 12 | 完整程式 2/3

12_strict_score.c

【教師講解】

本頁為 12_strict_score.c 的第 2/3 段，對應原始檔第 16~29 行。所有程式頁依序接續，沒有省略程式碼。略過尾端空白時把字元轉成 unsigned char 再交給 isspace。全部合法才寫入 *out。主程式以 fgets 控制輸入長度。

【程式帶讀】

對應 12_strict_score.c，原始碼第 16~29 行。這是完整檔案第 2/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
while (isspace((unsigned char)*end)) ++end;
if (*end != '\0') return TRAILING;
*out = (int)value;
return OK;
}

int main(void) {
    char line[128];
    int score;
    if (fgets(line, sizeof line, stdin) == NULL) {
        fputs(ferror(stdin) ? "Read error\n" : "No input\n", stderr);
        return EXIT_FAILURE;
    }
    if (strchr(line, '\n') == NULL && !feof(stdin)) {
```

【課堂提問】

為何 *out=(int)value 要放在所有檢查之後？

【參考答案】

只有值已落在 0~100，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

【來源與增補】

B pp.103–104,132–135 | C pp.6–9 | 新增

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 104 頁 | 案例 12 | 完整程式 3/3

【本頁目標】

案例 12 | 完整程式 3/3

12_strict_score.c

【教師講解】

本頁為 12_strict_score.c 的第 3/3 段，對應原始檔第 30~43 行。所有程式頁依序接續，沒有省略程式碼。主程式拒絕過長或讀取失敗的資料，依狀態回報可理解的錯誤文字。只有 OK 才使用 score；不要把有效輸出值和錯誤碼混用。

【程式帶讀】

對應 12_strict_score.c，原始碼第 30~43 行。這是完整檔案第 3/3 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
fputs("Line too long (max 126 bytes)\n", stderr);
return EXIT_FAILURE;
}
enum Status status = parse_score(line, &score);
if (status != OK) {
    const char *message[] = {
        "OK", "No number", "Trailing text", "Out of range"
    };
    fprintf(stderr, "%s\n", message[status]);
    return EXIT_FAILURE;
}
printf("score=%d\n", score);
return EXIT_SUCCESS;
}
```

【課堂提問】

為何 *out=(int)value 要放在所有檢查之後？

【參考答案】

只有值已落在 0~100，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

【來源與增補】

B pp.103–104,132–135 | C pp.6–9 | 新增

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 105 頁 | 案例 12 | 測試與常見錯誤

【本頁目標】

案例 12 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。atoi 無法清楚區分「合法 0」與轉換失敗；只看 errno 也無法偵測 86abc。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

一般 | +86 | score=86 | 允許前後空白

尾端垃圾 | 86abc | Trailing text | 不能只接受數字前綴

超大整數 | 60 個 9 | Out of range | 使用 ERANGE

空白行 | 只有空白 / 換行 | No number | end==text

超長一行 | 200 個字元 | Line too long | 不可默默截斷

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

為何 *out=(int)value 要放在所有檢查之後？

【參考答案】

只有值已落在 0 ~ 100，且沒有尾端垃圾時才承諾成功；如此失敗不會把半合法結果交給呼叫端。

【來源與增補】

B pp.103–104,132–135 | C pp.6–9 | 新增

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 106 頁 | assert | 記錄內部不變量，不替代外部驗證

【本頁目標】

assert | 記錄內部不變量，不替代外部驗證

NDEBUG 會讓 assert 的運算式不被執行；裡面不能放必要副作用。

【教師講解】

將輸入邊界 if 與內部檢查 assert 並列。原稿建議完成後關閉 assert，本教材不把它當作一律適用的規則；是否停用應依專案需求與測試政策。核心要求是即使 assert 被停用，必要的輸入、I/O 與資源處理仍存在。assert 失敗通常會輸出診斷並呼叫 abort，不適合期待程式正常恢復。

【本頁收斂】

assert(expr) 中的 expr 應是無必要副作用的檢查；發布政策不等於輸入驗證政策。

【課堂提問】

assert(fclose(fp)==0) 為何危險？

【參考答案】

因為定義 NDEBUG 時，fclose 可能根本不會執行。必要的資源釋放不能放在 assert 裡。

【來源與增補】

C pp.3–5

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 107 頁 | 案例 13 | 外部驗證與 assert 的分工

【本頁目標】

案例 13 | 外部驗證與 assert 的分工

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。讀入數值後計算平方根。外部不合法輸入必須以正常錯誤路徑拒絕；assert 僅記錄驗證後的內部不變量。只接受有限且非負的值；NaN、正負無窮大與負數拒絕。定義 NDEBUG 後，外部驗證仍須有效。

【完整規格】

任務：讀入數值後計算平方根。外部不合法輸入必須以正常錯誤路徑拒絕；assert 僅記錄驗證後的內部不變量。

輸入：標準輸入：一個可由 double 表示的數值文字。

輸出：有效：輸出 sqrt=結果（小數 6 位）；無效：stderr 訊息並失敗。

限制：只接受有限且非負的值；NaN、正負無窮大與負數拒絕。定義 NDEBUG 後，外部驗證仍須有效。

範例輸入：

4

範例輸出：

sqrt=2.000000

【課堂提問】

assert 關閉之後，這個程式會接受負數嗎？

【參考答案】

不會。外部驗證在 if 中，與 assert 是否啟用無關；assert 只是協助檢查內部邏輯。

【來源與增補】

C pp.3–5 | 新增防護

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 108 頁 | 案例 13 | 核心解法與圖解

【本頁目標】

案例 13 | 核心解法與圖解

外部驗證與 assert 的分工

【教師講解】

依下列順序帶讀圖解：1. if 檢查轉換結果、isfinite(x) 與 $x \geq 0$ 。2. assert($x \geq 0$) 說明程式到這裡應滿足的條件。3. 用一般版與 -DNDEBUG 版跑同一批測試。不可寫 `assert(scanf(...)==1)`；NDEBUG 啟用時，連讀取輸入的副作用都可能消失。

【課堂提問】

assert 關閉之後，這個程式會接受負數嗎？

【參考答案】

不會。外部驗證在 if 中，與 assert 是否啟用無關；assert 只是協助檢查內部邏輯。

【來源與增補】

C pp.3-5 | 新增防護

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 109 頁 | 案例 13 | 完整程式 1/1

【本頁目標】

案例 13 | 完整程式 1/1

13_assert_sqrt.c

【教師講解】

本頁為 13_assert_sqrt.c 的第 1/1 段，對應原始檔第 1 ~ 15 行。所有程式頁依序接續，沒有省略程式碼。先用 if 檢查轉換成功、有限值與非負條件，再以無副作用的 assert 驗證內部不變條件，最後計算 sqrt。加入 -DNDEBUG 之後，外部驗證仍然執行。

【程式帶讀】

對應 13_assert_sqrt.c，原始碼第 1 ~ 15 行。這是完整檔案第 1/1 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <assert.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    double x;
    if (scanf("%lf", &x) != 1 || !isfinite(x) || x < 0.0) {
        fputs("Expected a finite, nonnegative value\n", stderr);
        return EXIT_FAILURE;
    }
    assert(x >= 0.0); /* Internal invariant; no side effects. */
    printf("sqrt=%.6f\n", sqrt(x));
    return EXIT_SUCCESS;
}
```

【課堂提問】

assert 關閉之後，這個程式會接受負數嗎？

【參考答案】

不會。外部驗證在 if 中，與 assert 是否啟用無關；assert 只是協助檢查內部邏輯。

【來源與增補】

C pp.3-5 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 110 頁 | 案例 13 | 測試與常見錯誤

【本頁目標】

案例 13 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。不可寫 `assert(scanf(...)==1)`；NDEBUG 啟用時，連讀取輸入的副作用都可能消失。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

有效 | 4 | `sqrt=2.000000` | 正常路徑

邊界 | 0 | `sqrt=0.000000` | 合法輸入

負數 | -1 | 一般錯誤返回 | 不依賴 `abort`

特殊值 | `nan / inf` | 失敗 | 有限性檢查

移除 `assert` | -DNDEBUG 後輸入 -1 | 仍失敗 | 輸入驗證不可消失

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

`assert` 關閉之後，這個程式會接受負數嗎？

【參考答案】

不會。外部驗證在 `if` 中，與 `assert` 是否啟用無關；`assert` 只是協助檢查內部邏輯。

【來源與增補】

C pp.3-5 | 新增防護

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 111 頁 | 數學錯誤 | 先確認回報機制，再解釋結果

【本頁目標】

數學錯誤 | 先確認回報機制，再解釋結果

函式庫診斷、浮點特殊值與一般運算子，不能全部混成 `errno`。

【教師講解】

原稿把 1.0/0.0 與數學函式列在同一 `errno` 測試中，這不構成可攜的錯誤碼示範。改用 `log` 的文檔化行為，並依 `math_errhandling` 確認 `errno` 機制。NaN/無窮大表示也有實作相關條件；本教材只對平台承諾的分類做測試。

【課堂提問】

看到 `errno=0`，能推論任何浮點計算都成功嗎？

【參考答案】

不行。一般運算子不會保證設定 `errno`，數學函式也可能採浮點例外旗標等機制。

【來源與增補】

C pp.7-9 | W1 §7.12.1

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 112 頁 | 案例 14 | 觀察 log 的 errno 診斷

【本頁目標】

案例 14 | 觀察 log 的 errno 診斷

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。依序計算 $\log(-1)$ 、 $\log(0)$ 、 $\log(1)$ ，觀察數學函式的結果與錯誤分類，不把平台輸出字串當成固定答案。只有 `math_errhandling` 含 `MATH_ERRNO` 時，才以 `errno` 驗證錯誤分類。`NaN/inf` 文字與 `strerror` 訊息依實作。

【完整規格】

任務：依序計算 $\log(-1)$ 、 $\log(0)$ 、 $\log(1)$ ，觀察數學函式的結果與錯誤分類，不把平台輸出字串當成固定答案。

輸入：無外部輸入；程式內的 `input` 陣列固定為 `-1`、`0`、`1`。

輸出：先印 `MATH_ERRNO` 是否支援；逐筆印結果與 `EDOM`、`ERANGE`、`none`，或不支援 `errno` 檢查。

限制：只有 `math_errhandling` 含 `MATH_ERRNO` 時，才以 `errno` 驗證錯誤分類。`NaN/inf` 文字與 `strerror` 訊息依實作。

範例輸入：

直接執行 `math_errno`

範例輸出：

支援 `MATH_ERRNO` 時：

`-1` → `EDOM`；`0` → `ERANGE`；`1` → `none`

【課堂提問】

如果不在每次 `log` 前清除 `errno`，會發生什麼誤判？

【參考答案】

先前失敗留下的非零值可能仍存在，後續成功呼叫就會被誤認為失敗；仍須遵守特定函式的規格。

【來源與增補】

C pp.6–9 | 勘誤 / 補充

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 113 頁 | 案例 14 | 核心解法與圖解

【本頁目標】

案例 14 | 核心解法與圖解

觀察 log 的 errno 診斷

【教師講解】

依下列順序帶讀圖解：1. 先確認數學函式的錯誤回報機制。2. 每次呼叫 log 前把 errno 設為 0。3. log 後立即保存 errno，再交給 printf / strerror 產生說明。原稿的 1.0/0.0 不是會替你設定 errno 的函式呼叫；不能把它當作可攜的 ERANGE 示範。

【課堂提問】

如果不在每次 log 前清除 errno，會發生什麼誤判？

【參考答案】

先前失敗留下的非零值可能仍存在，後續成功呼叫就會被誤認為失敗；仍須遵守特定函式的規格。

【來源與增補】

C pp.6-9 | 勘誤 / 補充

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 114 頁 | 案例 14 | 完整程式 1/2

【本頁目標】

案例 14 | 完整程式 1/2

14_math_errno.c

【教師講解】

本頁為 14_math_errno.c 的第 1/2 段，對應原始檔第 1 ~ 13 行。所有程式頁依序接續，沒有省略程式碼。先查詢 math_errhandling 是否支援 MATH_ERRNO，再逐項測試 -1、0、1。每次呼叫 log 之前都清零 errno。

【程式帶讀】

對應 14_math_errno.c，原始碼第 1 ~ 13 行。這是完整檔案第 1/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <errno.h>
#include <math.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    const double input[] = {-1.0, 0.0, 1.0};
    int has_errno = (math_errhandling & MATH_ERRNO) != 0;
    printf("MATH_ERRNO=%d\n", has_errno);
    for (size_t i = 0; i < sizeof input / sizeof input[0]; ++i) {
        errno = 0;
        double result = log(input[i]);
        int saved_errno = errno;
```

【課堂提問】

如果不在每次 log 前清除 errno，會發生什麼誤判？

【參考答案】

先前失敗留下的非零值可能仍存在，後續成功呼叫就會被誤認為失敗；仍須遵守特定函式的規格。

【來源與增補】

C pp.6–9 | 勘誤 / 補充

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 115 頁 | 案例 14 | 完整程式 2/2

【本頁目標】

案例 14 | 完整程式 2/2

14_math_errno.c

【教師講解】

本頁為 14_math_errno.c 的第 2/2 段，對應原始檔第 14~26 行。所有程式頁依序接續，沒有省略程式碼。立即保存 log 呼叫之後的 errno，再輸出結果與診斷。未支援時明確說明不能用 errno 檢查；診斷字串和 NaN / 無限大顯示方式不作跨平台固定答案。

【程式帶讀】

對應 14_math_errno.c，原始碼第 14~26 行。這是完整檔案第 2/2 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
printf("x=%.0f result=%g ", input[i], result);
if (!has_errno)
    puts("errno checking not supported");
else if (saved_errno == 0)
    puts("error=none");
else {
    const char *name = saved_errno == EDOM ? "EDOM" :
        saved_errno == ERANGE ? "ERANGE" : "OTHER";
    printf("error=%s (%s)\n", name, strerror(saved_errno));
}
}
return 0;
}
```

【課堂提問】

如果不在每次 log 前清除 errno，會發生什麼誤判？

【參考答案】

先前失敗留下的非零值可能仍存在，後續成功呼叫就會被誤認為失敗；仍須遵守特定函式的規格。

【來源與增補】

C pp.6–9 | 勘誤 / 補充

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 116 頁 | 案例 14 | 測試與常見錯誤

【本頁目標】

案例 14 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、stdout、stderr 與檔案內容。原稿的 1.0/0.0 不是會替你設定 errno 的函式呼叫；不能把它當作可攜的 ERANGE 示範。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

定義域錯誤 | $\log(-1)$ | EDOM (支援時) | 參數超出實數定義域

極點錯誤 | $\log(0)$ | ERANGE (支援時) | 零點的發散

正常 | $\log(1)$ | 結果 0 ; none | 清除先前殘留值

平台差異 | MATH_ERRNO 未設 | 明確標示不支援 | 不可宣稱 `errno=0` 即安全

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

如果不在每次 `log` 前清除 `errno`，會發生什麼誤判？

【參考答案】

先前失敗留下的非零值可能仍存在，後續成功呼叫就會被誤認為失敗；仍須遵守特定函式的規格。

【來源與增補】

C pp.6–9 | 勘誤 / 補充

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 117 頁 | 清理流程 | 失敗可以提早發生，資源仍要正確歸還

【本頁目標】

清理流程 | 失敗可以提早發生，資源仍要正確歸還

把「拿到了哪些資源」當成狀態，而不是到處複製 fclose。

【教師講解】

對照來源未開啟、來源已開啟、兩檔已開啟、已完成寫入四種狀態。每個失敗分支只清理自己真正取得的資源。對新建檔案可以設計移除不完整結果，但 remove 可能失敗，原地更新也不等於交易式還原。關閉後不再使用 FILE*，避免重複關閉或使用失效串流。

【本頁收斂】

每個成功取得的資源，在每條離開路徑上都應被處理一次。

【課堂提問】

目的檔開啟失敗時，要不要刪除同名目的檔？

【參考答案】

不要；它可能是原本就存在的重要資料。只有確認是本次成功建立的檔案，才可依契約清理。

【來源與增補】

B pp.32–34,126–128 | C 綜合

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 118 頁 | 介面設計 | 把狀態碼與資料值分開

【本頁目標】

介面設計 | 把狀態碼與資料值分開

原稿 `find_int` 回傳 `-1/-2/-3` 作錯誤碼，可能和合法負數資料混淆。

【教師講解】

保留原稿 `find_int` 的動機，但指出其回傳碼設計的前提。若檔案中的整數允許為負值，`-1` 到 `-3` 就可能是合法資料，呼叫端無法分辨。可改用 `enum` 狀態搭配 `int*` 輸出參數；案例 12 的 `parse_score` 就使用同樣模式。此頁為介面設計片段，不是新增未提供程式的實作題。

【課堂提問】

資料本身可能是 `-1`，還能用 `return -1` 表示開檔失敗嗎？

【參考答案】

除非契約明確排除資料值 `-1`，否則應把成功 / 失敗狀態和資料通道分開。

【來源與增補】

B pp.115–116 | 新增介面補強

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 119 頁 | 案例 15 | 整合專題：成績檔驗證與報表

【本頁目標】

案例 15 | 整合專題：成績檔驗證與報表

完整題目：先確認任務、輸入、輸出與限制

【教師講解】

先不要展示解答。請學生用自己的話重述任務，再指出輸入格式、輸出格式與無效資料的處理。從成績文字檔載入資料，全部驗證通過後才建立統計報表；任一筆不合法就停止，不產生看似成功的一部分報表。1 ~ 100 筆；ID 為 S000 ~ S999 且不可重複；成績 0 ~ 100。無 BOM/NUL 的文字檔；禁止空白行；目的報表必須不存在。

【完整規格】

任務：從成績文字檔載入資料，全部驗證通過後才建立統計報表；任一筆不合法就停止，不產生看似成功的一部分報表。

輸入：命令列：grade_file input_csv new_report。每行 Sddd,score。沒有標題列。

輸出：報表第一行列出 count、average、min、max、pass；後續保留輸入順序，逐列輸出 ID、成績、PASS/FAIL。

限制：1 ~ 100 筆；ID 為 S000 ~ S999 且不可重複；成績 0 ~ 100。無 BOM/NUL 的文字檔；禁止空白行；目的報表必須不存在。

範例輸入：

S001,86

S002,100

S003,59

S004,75

範例輸出：

count=4 average=80.00 min=59 max=100 pass=3

S001,86,PASS

S002,100,PASS

S003,59,FAIL

S004,75,PASS

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 120 頁 | 專題契約 | 先說清楚哪些資料要拒絕

【本頁目標】

專題契約 | 先說清楚哪些資料要拒絕

每一條驗證規則，都應對應一個可重現的測試。

【教師講解】

讓學生區分語法規則、數值規則、容量規則與副作用規則。本題不是一般 CSV 解析器；不支援引號、欄位內逗號或標題列。這些限制明確寫出後，才有一致的驗收標準。

【表格帶讀】

項目 | 接受條件 | 失敗政策

來源格式 | 無標題列；Sddd,score；ID 前不可加空白 | 空白行、BOM、錯誤欄位立即失敗

分數文字 | 允許前後空白與正號；完整整數 0~100 | 86abc、小數、超大整數均拒絕

記錄數量 | 1~100 筆；ID 不得重複 | 第 101 筆或重複 ID，指出行號

單行長度 | 最多 126 bytes；來源為無 NUL 文字 | 拒絕被緩衝區截斷的一行

報表建立 | 全部輸入驗證、讀取與關閉成功之後 | 使用 wx；不得覆寫既有報表

【課堂提問】

檔案裡已有 99 筆，下一筆 ID 重複，應先存入再標示錯誤嗎？

【參考答案】

不應。先驗證 ID 和容量，成功才寫入 ids[n]/scores[n] 並增加 n。

【來源與增補】

A+B+C | 新增整合實作

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 121 頁 | 案例 15 | 核心解法與圖解

【本頁目標】

案例 15 | 核心解法與圖解

整合專題：成績檔驗證與報表

【教師講解】

依下列順序帶讀圖解：1. 載入：fgets 逐行取得資料，紀錄實際行號。2. 驗證：格式、strtol、範圍、ID 去重與容量全部通過才存入。3. 輸出：先完成所有驗證，再以 wx 建立報表；逐筆檢查寫入與關閉。先輸出報表再驗證剩餘輸入，會留下半份結果。本題先完整載入與驗證，再建立目的檔。

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 122 頁 | 專題資料模型 | 三個陣列，三種責任

【本頁目標】

專題資料模型 | 三個陣列，三種責任

儲存資料、記錄狀態、承接輸入，不要混成同一件事。

【教師講解】

對照四筆範例，ids 與 scores 以相同索引表示同一筆資料；seen 的索引卻是數值化 ID，長度因此為 1000。line 僅是可重複使用的暫存緩衝區，不是所有資料的永久存放處。

【課堂提問】

seen[86] 代表成績 86 分已經出現過嗎？

【參考答案】

不是；seen 的索引是 ID，seen[86] 代表 S086 已經被接受。

【來源與增補】

A+B+C | 新增整合實作

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 123 頁 | 案例 15 | 完整程式 1/6

【本頁目標】

案例 15 | 完整程式 1/6

15_grade_file.c

【教師講解】

本頁為 15_grade_file.c 的第 1/6 段，對應原始檔第 1 ~ 18 行。所有程式頁依序接續，沒有省略程式碼。匯入必要標頭與容量定義。parse_record 先檢查長度、S 前綴與逗號位置，再驗證三位 ID 數字，避免在未驗證前就使用索引。

【程式帶讀】

對應 15_grade_file.c，原始碼第 1 ~ 18 行。這是完整檔案第 1/6 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
#include <ctype.h>
#include <errno.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define CAP 100

static int parse_record(const char *line, int *id, int *score) {
    if (strlen(line) < 6 || line[0] != 'S' || line[4] != ',')
        return 0;
    for (int i = 1; i <= 3; ++i)
        if (!isdigit((unsigned char)line[i])) return 0;
    int number = (line[1] - '0') * 100 +
                (line[2] - '0') * 10 + line[3] - '0';
    char *end;
    errno = 0;
    long value = strtol(line + 5, &end, 10);
```

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼摺 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 124 頁 | 案例 15 | 完整程式 2/6

【本頁目標】

案例 15 | 完整程式 2/6

15_grade_file.c

【教師講解】

本頁為 15_grade_file.c 的第 2/6 段，對應原始檔第 19~36 行。所有程式頁依序接續，沒有省略程式碼。strtol 解析分數，檢查沒有數字、ERANGE、0~100 及尾端雜字；全部合法才把 ID 和分數交給呼叫端。主程式另驗證命令列。

【程式帶讀】

對應 15_grade_file.c，原始碼第 19~36 行。這是完整檔案第 2/6 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
if (end == line + 5 || errno == ERANGE || value < 0 || value > 100)
    return 0;
while (isspace((unsigned char)*end)) ++end;
if (*end != '\0') return 0;
*id = number;
*score = (int)value;
return 1;
}
```

```
int main(int argc, char *argv[]) {
    if (argc != 3) {
        fputs("usage: grade_file input_csv new_report\n", stderr);
        return EXIT_FAILURE;
    }
    FILE *in = fopen(argv[1], "r");
    if (in == NULL) {
        fputs("Cannot open input\n", stderr);
        return EXIT_FAILURE;
    }
}
```

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼摺 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 125 頁 | 案例 15 | 完整程式 3/6

【本頁目標】

案例 15 | 完整程式 3/6

15_grade_file.c

【教師講解】

本頁為 15_grade_file.c 的第 3/6 段，對應原始檔第 37 ~ 53 行。所有程式頁依序接續，沒有省略程式碼。開啟輸入後逐行讀取，維護行號。長行或 parse_record 失敗立即標記錯誤；只有完整解析的值才進入容量與重複檢查。

【程式帶讀】

對應 15_grade_file.c，原始碼第 37 ~ 53 行。這是完整檔案第 3/6 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
}
int ids[CAP], scores[CAP], n = 0, failed = 0;
bool seen[1000] = {false};
char line[128];
size_t line_no = 0;
while (fgets(line, sizeof line, in) != NULL) {
    ++line_no;
    int id, score;
    if (strchr(line, '\n') == NULL && !feof(in)) {
        fprintf(stderr, "Line %zu: too long\n", line_no);
        failed = 1;
        break;
    }
    if (!parse_record(line, &id, &score)) {
        fprintf(stderr, "Line %zu: invalid record\n", line_no);
        failed = 1;
        break;
    }
}
```

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼
碼摺記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確
差異 [跳至主要內容](#)

第 126 頁 | 案例 15 | 完整程式 4/6

【本頁目標】

案例 15 | 完整程式 4/6

15_grade_file.c

【教師講解】

本頁為 15_grade_file.c 的第 4/6 段，對應原始檔第 54~70 行。所有程式頁依序接續，沒有省略程式碼。透過 seen[id] 偵測重複 ID，在寫入 ids[n] 和 scores[n] 之前檢查容量。讀取結束後檢查 ferror 和 fclose；輸入失敗時仍不建立報表。

【程式帶讀】

對應 15_grade_file.c，原始碼第 54~70 行。這是完整檔案第 4/6 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
}
if (seen[id] || n == CAP) {
    fprintf(stderr, "Line %zu: duplicate ID or capacity exceeded\n",
        line_no);
    failed = 1;
    break;
}
seen[id] = true;
ids[n] = id;
scores[n] = score;
++n;
}
if (ferror(in)) {
    fputs("Input read error\n", stderr);
    failed = 1;
}
if (fclose(in) != 0) failed = 1;
```

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼摺 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 127 頁 | 案例 15 | 完整程式 5/6

【本頁目標】

案例 15 | 完整程式 5/6

15_grade_file.c

【教師講解】

本頁為 15_grade_file.c 的第 5/6 段，對應原始檔第 71 ~ 87 行。所有程式頁依序接續，沒有省略程式碼。確認至少一筆資料，再走訪有效陣列計算總和、極值與及格數。以 wx 建立全新的輸出檔；既有檔案保持不變。

【程式帶讀】

對應 15_grade_file.c，原始碼第 71 ~ 87 行。這是完整檔案第 5/6 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
if (failed) return EXIT_FAILURE;
if (n == 0) {
    fputs("Empty input\n", stderr);
    return EXIT_FAILURE;
}
int sum = 0, pass = 0, low = scores[0], high = scores[0];
for (int i = 0; i < n; ++i) {
    sum += scores[i];
    if (scores[i] >= 60) ++pass;
    if (scores[i] < low) low = scores[i];
    if (scores[i] > high) high = scores[i];
}
FILE *out = fopen(argv[2], "wx");
if (out == NULL) {
    fputs("Report exists or cannot be created\n", stderr);
    return EXIT_FAILURE;
}
```

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異 [跳至主要內容](#)

第 128 頁 | 案例 15 | 完整程式 6/6

【本頁目標】

案例 15 | 完整程式 6/6

15_grade_file.c

【教師講解】

本頁為 15_grade_file.c 的第 6/6 段，對應原始檔第 88~104 行。所有程式頁依序接續，沒有省略程式碼。檢查統計與逐筆 fprintf，也檢查 fclose。任何輸出失敗嘗試移除本次建立的部分報表；成功才回傳成功狀態。這不提供所有檔案系統的原子交易保證。

【程式帶讀】

對應 15_grade_file.c，原始碼第 88~104 行。這是完整檔案第 6/6 段，依頁序合併即可，不需補寫省略片段。提醒學生先讀題目頁的輸入資料前提；程式使用的語法片段與整體錯誤契約應一起理解。

```
if (fprintf(out, "count=%d average=%.2f min=%d max=%d pass=%d\n",
    n, (double)sum / n, low, high, pass) < 0)
    failed = 1;
for (int i = 0; i < n && !failed; ++i)
    if (fprintf(out, "%03d,%d,%s\n", ids[i], scores[i],
        scores[i] >= 60 ? "PASS" : "FAIL") < 0)
        failed = 1;
if (fclose(out) != 0) failed = 1;
if (failed) {
    fputs("Report write failed\n", stderr);
    if (remove(argv[2]) != 0)
        fputs("Warning: partial report remains\n", stderr);
    return EXIT_FAILURE;
}
puts("Report created");
return EXIT_SUCCESS;
}
```

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

ids/scores 保存最多 100 筆資料；seen[1000] 記錄 ID 是否出現；line[128] 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼摺 [跳至主要內容](#) 記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

第 129 頁 | 案例 15 | 測試與常見錯誤

【本頁目標】

案例 15 | 測試與常見錯誤

先預測結果，再執行；不只測一般資料。

【教師講解】

依表格逐組測試，確認返回狀態、`stdout`、`stderr` 與檔案內容。先輸出報表再驗證剩餘輸入，會留下半份結果。本題先完整載入與驗證，再建立目的檔。對隨機或平台相依輸出，應測試性質，而不是把單一次執行結果當成唯一答案。

【測試判讀】

正常 | 題目四筆資料 | 平均 80.00；及格 3 | 檔案→陣列→報表

重複 ID | S001 出現兩次 | 失敗；不產生報表 | `seen[1000]` 去重

格式 / 數值 | 86abc、101、超大整數 | 失敗並指出行號 | 拒絕不完整解析

空檔 / 超量 | 0 筆或 101 筆 | 失敗；不產生報表 | 有效範圍 1 ~ 100

目的已存在 | 已存在的報表 | 失敗；原檔不變 | 建立模式保護資料

本頁列出教學代表測試。附件測試器另外包含一般值、邊界與錯誤路徑；未模擬磁碟滿載、斷電、併發或所有作業系統。

【課堂提問】

這個專題在哪三個位置直接使用陣列？

【參考答案】

`ids/scores` 保存最多 100 筆資料；`seen[1000]` 記錄 ID 是否出現；`line[128]` 承接每一行文字。

【來源與增補】

A+B+C | 新增整合實作

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 130 頁 | 專題結果 | 正常輸出與失敗時的檔案狀態

【本頁目標】

專題結果 | 正常輸出與失敗時的檔案狀態

同樣重要的兩個答案：算出什麼，以及哪些檔案不該被改動。

【教師講解】

用四筆樣本逐筆重算總分 320、平均 80、及格 3。接著示範在第二行加入非法分數時，報表不應被建立；如果目的報表早已存在，內容應維持不變。這比只看終端機印出成功更能驗證整合行為。

【課堂提問】

輸入驗證全部通過，就可以在寫入前印出 Report created 嗎？

【參考答案】

不能。fprintf 或 fclose 仍可能失敗；完成訊息必須放在所有必要輸出成功之後。

【來源與增補】

A+B+C | 新增整合實作

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 131 頁 | 課堂檢核 | 先判斷問題，再提出最小修正

【本頁目標】

課堂檢核 | 先判斷問題，再提出最小修正

請先說明理由；教師備忘錄附參考答案。

【教師講解】

本頁可以作為分組討論或隨堂短答。第一題 `i=N` 越界，改 `i<N`。第二題以 `fread` 回傳控制，再檢查 `ferror`。第三題輸入副作用可能消失，應改 `if` 再視需要 `assert`。第四題改用本次有效長度 `n`。第五題回傳值是主要契約，`errno` 可能是殘留值。請學生先提出原因，再寫修正，避免只背「標準答案」。

【表格帶讀】

片段 / 說法 | 要回答的問題 | 討論方向

`for(i=0; i<=N; ++i) a[i]=0;` | 哪一次迭代越界？ | 索引與長度

`while(!feof(fp)) fread(...);` | 為何不能預測下一次讀取？ | 回傳值與狀態

`assert(scanf("%d",&n)==1);` | NDEBUG 後有何差異？ | 副作用與外部驗證

`fwrite(buf,1,sizeof buf,out);` | 最後一塊只有 1 byte 時如何？ | 容量與有效長度

`errno != 0` 就宣告任務失敗 | 為何可能誤判？ | 診斷與實際結果

【課堂提問】

這五個問題的共同根源是什麼？

【參考答案】

把不同概念當成同一件事：長度與索引、預測與狀態、檢查與副作用、容量與有效長度、診斷與成功契約。

【來源與增補】

A+B+C | 新增檢核

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 132 頁 | 交作業前 | 讓「可執行」升級為「可驗證」

【本頁目標】

交作業前 | 讓「可執行」升級為「可驗證」

把程式、測試與說明當成同一份交付內容。

【教師講解】

使用這一頁做作業驗收，不只看能不能編譯。特別要求同一份資料的人工預期結果，和失敗情況下的檔案狀態。程式需要清楚標示受支援的輸入範圍與平台限制；沒有進行的故障測試也應坦白列出，不能把正常執行當成所有錯誤都已驗證。

【課堂提問】

安全複製器除了比對輸出檔，還應測什麼？

【參考答案】

至少測目的檔已存在、來源與目的相同、空檔、區塊邊界及來源不存在；並確認失敗時原檔沒有被改動。

【來源與增補】

A+B+C | 新增驗收表

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 133 頁 | 附錄 A | printf 常用轉換與長度修飾子

【本頁目標】

附錄 A | printf 常用轉換與長度修飾子

需要細節時回查；不必把所有規格放在主講解頁。

【教師講解】

附錄保留原稿完整規格查詢功能，而不是逐頁列一兩個項目。整數長度與有號性要一起看；對 z/t 修飾子的有號對應型別，不能簡化成所有轉換都直接要求同一個型別。%n 屬特殊用途，必須傳入正確指標，且不應使用外部輸入作格式字串。

【表格帶讀】

格式類別 | 對應型別 / 意義 | 注意

d/i ; o/u/x/X | int ; unsigned int | 進位表示與符號規則不同

f/e/g/a | double ; 小數 / 科學 / 有效數字 / 十六進位浮點 | float 透過可變參數時提升為 double

hh/h/l/ll | char/short 經提升 ; long ; long long | 配 d/u/x 等選擇相應整數長度

j / z / t | intmax_t ; size_t ; ptrdiff_t 相關 | 修飾子需配合實際 signed/unsigned 轉換

L + 浮點轉換 | long double | 例如 %Lf

c / s / p / n / % | 字元 / 字串 / 位址 / 記錄已輸出數 / 百分號 | %n 寫入物件，不輸出字元

【課堂提問】

printf("%n", &n) 會輸出 n 的值嗎？

【參考答案】

不會。它把本次呼叫到此為止已輸出的字元數寫進 n。

【來源與增補】

B pp.67–85

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 134 頁 | 附錄 B | scanf 的型別與格式語意

【本頁目標】

附錄 B | scanf 的型別與格式語意

輸入函式操作物件位址，格式不匹配可能造成未定義行為。

【教師講解】

用作課後索引表。原稿完整枚舉的 j/z/t 等修飾子，主教材已說明型別匹配原則；實際使用時仍應按函式規格查對有號性。特別提醒 %u 不等於「拒絕負號」，輸入合法性要由規格另外保證或以 strtol 解析後檢查。

【表格帶讀】

格式 / 修飾子 | 接收物件 | 提醒

%d / %i | int* | %i 依前綴辨別進位；%d 固定十進位

%u / %o / %x | unsigned int* | 仍可能接受符號，不能代替非負驗證

%f / %lf / %Lf | float* / double* / long double* | 與 printf 的參數提升不同

%hhd / %hd / %ld / %lld | signed char* / short* / long* / long long* | 目標值必須可由接收型別表示

%s / %[/ %c | char 陣列 / 字元物件 | %s 與 %[補 \0；%c 不補

%* / %n | 抑制指定 / int* 計數 | 兩者都不增加回傳的指定數

【課堂提問】

使用 %u 就可以保證使用者沒有輸入負數嗎？

【參考答案】

不能。格式可能接受符號並依無號轉換規則處理；要拒絕負數必須另外驗證。

【來源與增補】

B pp.95–109

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

[跳至主要內容](#)

第 135 頁 | 附錄 C | 明確標示原稿勘誤與安全補強 (1/2)

【本頁目標】

附錄 C | 明確標示原稿勘誤與安全補強 (1/2)

以本教材的 C17 基準解讀；不是把平台實驗結果當成語言保證。

【教師講解】

此頁集中保留修正紀錄，讓授課者可追溯到原始投影片，避免學生回去對照原稿時困惑。修正的基準是本教材採用的 C17 與實際可驗證格式輸出，不把差異悄悄隱藏在重排內容中。

【表格帶讀】

原稿位置 | 需要釐清的內容 | 本教材處理

A p.16 | 空初始列的合法性敘述 | C17 使用 {0}，不採 {}

A p.27 | C++ 的 std::size_t 圖片 | C 語言改說 size_t，輸出用 %zu

B p.13 / p.64 | stderr 一定顯示螢幕 | 限定為只重導向 stdout 的情境

B p.57 | 輸出後才 setvbuf | 先設定緩衝，再對該串流 I/O

B p.81 | %.4g 的 1234.56 結果 | 修正為 1235

【課堂提問】

為何要把原稿頁碼保留下來？

【參考答案】

方便追溯、比較和備課，能清楚區分原始概念、新增案例與勘誤。

【來源與增補】

A+B | 勘誤清單；W1

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 136 頁 | 附錄 C | 明確標示原稿勘誤與安全補強 (2/2)

【本頁目標】

附錄 C | 明確標示原稿勘誤與安全補強 (2/2)

保留原意，但不沿用會誤導學生的測試與安全假設。

【教師講解】

指出「補強」不一定表示原教材主題錯誤，而是教材示例缺少實務條件。例如原始 `fcopy` 說明主要資料移動，但未檢查所有輸出失敗；本教材以新的題目契約補上目的檔保護與不完整檔案處理。對於移植性限制，保留不同實作可能有不同結果的說明。

【表格帶讀】

原稿位置 | 需要釐清的內容 | 本教材處理

B pp.132–135 | `gets` 的歷史敘述 | C17 不使用；改 `fgets` 並檢查長行

B pp.128,143–146 | 漏檢查 I/O、未初始化欄位、`!feof` 迴圈 | 完整案例驗證有效長度與讀寫 / 關閉結果

B pp.115–116 | `find_int` 的負數錯誤碼 | 示範狀態與資料分離的介面

B pp.152–159 | 文字位移與 `fpos_t` 大檔推論 | 依串流類型與實作說明限制

C p.7 | 1.0/0.0 與 `errno` 的關聯 | 改用 `log` 並檢查 `math_errhandling`

C p.5 | 完成後一律停用 `assert` | 依專案策略；外部驗證不能依賴 `assert`

【課堂提問】

補上錯誤檢查之後，是否就能宣稱檔案寫入具備斷電交易安全？

【參考答案】

不能。這些檢查處理標準 I/O 回報的失敗，不等於作業系統與儲存裝置層的交易或持久化保證。

【來源與增補】

B+C | 勘誤 / 補強清單；W1

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>（C11 委員會草案，引用與 C17 共通的核心條款）

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 137 頁 | 來源與教材邊界

【本頁目標】

來源與教材邊界

原稿為內容主幹；新增示例與勘誤在各頁標示，備忘錄保留來源。

【教師講解】

三份使用者提供的原稿分別為 Chapter 8 Arrays(1).pptx (64 頁)、Chapter 22 InputOutput(1).pptx (172 頁)、Chapter 24 Error Handling(1).pptx (9 頁)。外部參考僅用於確認 C 語意與平台介面：W1 WG14 N1570 公開委員會草案、W2 Microsoft Learn C11/C17 支援說明、W3 Microsoft fopen 文件、W4 Microsoft strtol 文件。N1570 是公開 C11 委員會草案，本文引用的是與本教材 C17 共同的核心規則，不宣稱它是 ISO C17 正式版。

【課堂提問】

新增案例是否是原稿作者原有的題目？

【參考答案】

不是。新增案例、規格、圖解與測試為本次重整時補充；以標籤和來源頁碼區分。

【來源與增補】

A+B+C；W1–W4

A：Chapter 8 Arrays(1).pptx；B：Chapter 22 InputOutput(1).pptx；C：Chapter 24 Error Handling(1).pptx。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。

【外部校核來源】

W1 — <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf> (C11 委員會草案，引用與 C17 共通的核心條款)

W2 — <https://learn.microsoft.com/en-us/cpp/overview/install-c17-support?view=msvc-170>

W3 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen?view=msvc-170>

W4 — <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strtol-wcstol-strtol-l-wcstol-l?view=msvc-170>

[跳至主要內容](#)

第 138 頁 | 案例索引 | 題目、解法、程式與測試都在一起

【本頁目標】

案例索引 | 題目、解法、程式與測試都在一起

點選案例可跳回完整題目頁；附件提供同名獨立 .c 檔。

【教師講解】

本頁可用於課堂快速跳題。請始終先回到完整題目頁，避免只給學生看解答程式。附件有 16 支可獨立編譯的程式、測試資料、測試執行器與逐頁備忘錄；案例 10 的兩個版本接受同一個複製題目契約。

【課堂提問】

複習一題時應先看哪一頁？

【參考答案】

先看題目規格和輸入限制，再自行預測核心解法與測試，最後比對程式。

【來源與增補】

A+B+C | 新增索引

A : Chapter 8 Arrays(1).pptx ; B : Chapter 22 InputOutput(1).pptx ; C : Chapter 24 Error Handling(1).pptx 。頁碼指原始投影片。標記「新增」的題目規格、圖解、案例及測試是整合教學增補；標記「勘誤」處與原稿文字有明確差異。